

Contents

Architecture Deep Dive	2
1. From Architectural Idea to Runtime Mechanics	2
2. A Note on the Two Implementations	2
3. Composition Root	2
4. ShowcaseApp	3
5. Application Event Map	4
6. EventManager — Static Event Maps	5
7. EventManager — Dynamic Subscriptions	6
8. EventManager Contract Validation	7
9. Slider Prototype Hierarchy	7
10. BaseSlider — Core Responsibilities	8
11. BaseSlider — Movement Lifecycle	8
12. Why This Distinction Matters	9
13. PaginationSlider	10
14. KeyboardSlider	10
15. Keyboard Configuration as a Strategy	11
16. InfiniteSlider	11
17. Infinite Loop Teleport	12
18. Index Normalization	12
19. DraggableSlider — Lifecycle	13
20. Drag Threshold	13
21. AutoscrollerSlider	13
22. Autoscroller State Machine	14
23. Autoscroller Resume Logic	15
24. Autoscroller Wake-Up Timing	15
25. Automatic vs Manual Movement	16
26. AudioPlayer — Domain Boundary	16
27. Audio FSM	17
28. Audio Track and Album Indices	18
29. Audio Event Model	18
30. Slider <-> Audio Synchronization	19
31. Autoscroller <-> Audio Synchronization	19
32. Album Playback Locks Autoscroller	20
33. albumplaypassthrough	21
34. Album End	21
35. Input Modes in ShowcaseApp	21
36. Repeat Filtering	22
37. Reverse Pipeline	23
38. _pipe() Contract	23
39. Why This Resembles Chain of Responsibility	24
40. Inheritance + Parent Handler Cooperation	24
41. Template Method-Like Structure	25

42. Static Class Contracts	25
43. Module Constants vs Static Properties	26
44. this.constructor and Polymorphic Static Lookup	26
45. Read-Only Static Contracts	27
46. Shop and Default URL	27
47. ButtonManager and KeyboardManager Return Contract	27
48. Mouse and Button Pipeline	28
49. Auxiliary Mouse Input	28
50. Focus and Keyboard UI State	29
51. Autoscroll and UI Mode	29
52. Audio Mode and UI Mode	30
53. AudioDeckView	30
54. Audio Track Change	31
55. Audio Timeline	31
56. Media Session	32
57. DOM Validation as Explicit Contract	32
58. Why Fallback Selector Guessing Is Avoided	33
59. Resize Lifecycle	33
60. Dragging and Autoscroll Interaction	34
61. Audio and Autoscroll Are Separate FSMs	34
62. Configuration Structure	35
63. Configuration as a Contract	35
64. Micro-Framework Characteristics	35
65. What Is Intentionally Not Abstracted	36
66. Why the State Pattern Is Not Forced	36
67. Pattern Map — More Precise Classification	37
68. Static Inheritance in the Prototype Version	37
69. Alias Discipline	38
70. Prototype Version vs Class Version	39
71. The Architecture as a Small Internal Ecosystem	39
72. Typical User Interaction — Keyboard	40
73. Typical User Interaction — Drag	41
74. Typical User Interaction — Album Playback	41
75. Main Theme / Autoscroll Interaction	42
76. Data Flow	43
77. Component Contracts	43
78. Error Boundaries and Validation	44
79. Why the Architecture Is More Complex Than a Typical Slider	45
80. Architectural Trade-off	45
81. What Makes the Architecture Coherent	46
82. Architectural Core	46
83. Final Summary	47
84. Cross-Cutting Architectural Story	49
84.1 Why the Pieces Fit Together	49

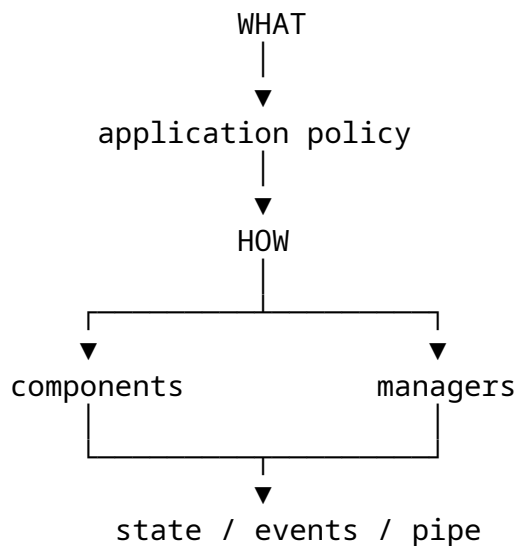
84.2 Why Managers Are More Than Convenience Wrappers	49
84.3 Why _pipe() Is Worth Documenting	50
84.4 Why Local FSMs Are Preferable Here	50
84.5 Why the Architecture Does Not Chase Patterns	50
84.6 The Educational Goal	51
84.7 That makes the project useful as a study object: the UI gives the architecture something concrete to control, while the architecture gives the UI a laboratory in which OOP and system-design ideas can be explored without hiding them behind a framework.	51
85. Timer Deep Dive: Why bootstrap Exists	51
86. How Timer.bootstrap Actually Works	52
87. Why bootstrap Belongs to Timer, Not AutoscrollSlider	53
88. bootstrap and Adaptive Autoscroll	53
89. Album End -> Slider -> slidechange -> AudioPlayer	54
90. Why the Feedback Loop Is Not a Design Failure	55
91. Album-End Event Is Cancelable	56
92. Active Document vs Background Document	56
93. A More Complete Synchronization Loop	57
94. Guard Conditions Prevent Unwanted Cycling	58
95. Album-End and the Separation of Intent	58
96. Event Pipelines and Event Publication Are Different Mechanisms	59
97. One Interaction Can Cross Both Models	60
98. Audio Track Change as a Similar Round Trip	60
99. Timer and Event Architecture Meet at Autoscroll	61
100. Media Session as Another Input Boundary	62
101. Why Preventing Default Browser Behavior Is Not the Whole Input Architecture	62
102. State Ownership Across the Feedback Loop	63
103. The Architecture Is Event-Driven Without Being Globally Event-Based	63
104. Lifecycle Ownership	64
105. Why These Small Objects Form a Coherent Architecture	64
106. A Note on the Project's Educational Evolution	65
107. Prototype/Class Parity as an Architectural Experiment	66
108. Final End-to-End Picture	66
109. Deep-Dive Closing Perspective	67

Architecture Deep Dive

1. From Architectural Idea to Runtime Mechanics

The Overview explains the architecture as a system of cooperating responsibilities. This document follows the same system down into implementation details. The important question remains the same throughout: **why does this piece exist, what does it own, and how does it cooperate with neighboring pieces without stealing their responsibility?**

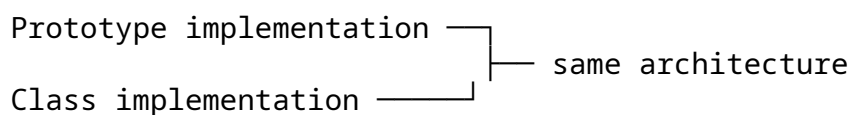
A useful mental model is:



The implementation contains many small decisions - static event maps, dynamic subscriptions, normalized indexes, polymorphic state lookup, temporary UI states, timing gates and media event translation. They are easier to understand when each is treated as a consequence of the larger architectural rule rather than as an isolated trick.

2. A Note on the Two Implementations

The project exists in prototype-based and ES6-class forms. The architecture is intentionally shared. The prototype version makes some JavaScript object-model mechanics more visible, while the class version expresses the same hierarchy and contracts using modern class syntax.

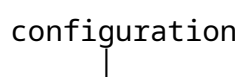


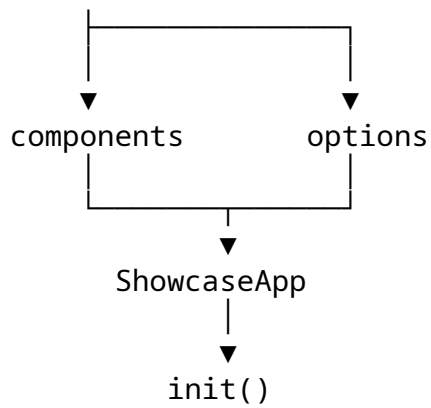
The detailed discussion therefore focuses on architectural behavior first and implementation syntax second.

3. Composition Root

`main.js` is responsible for assembling the application.

Conceptually:





ShowcaseApp receives:

```
slider
audioPlayer
audioDeckView
shop
options
```

and creates the application-level infrastructure it needs:

```
DOMValidator
KeyboardManager
EventManager
```

The object graph is therefore established from outside the components rather than being hidden inside unrelated methods.

This is the practical IoC/DI part of the design.

4. ShowcaseApp

ShowcaseApp is the highest-level runtime coordinator.

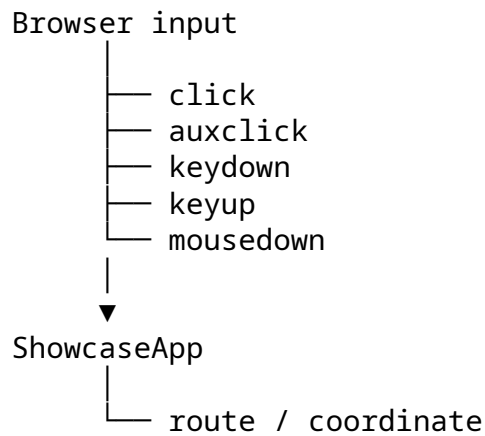
Its private state is deliberately small:

```
_slider
_audioPlayer
_audioDeckView
_shop
_options

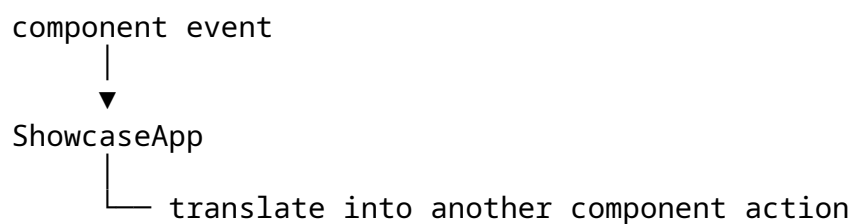
_domValidator
_keyboardManager
_eventManager

_isSliderMoving
_btnNoActive
```

The application-level responsibilities are:



and:



It does not implement the Slider's movement algorithm or AudioPlayer's playback algorithm.

5. Application Event Map

The main event map includes browser events and component-generated custom events.

Browser events

click
auxclick
keydown
keyup
mousedown

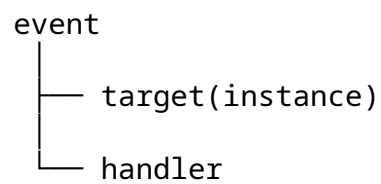
Slider events

viewportclick
slidemove
slidechange
autoscrollchange

Audio events

albumplay
albumpause
albumplaypassthrough
albumend
audiotrackchange
timechange

The event map is declarative:



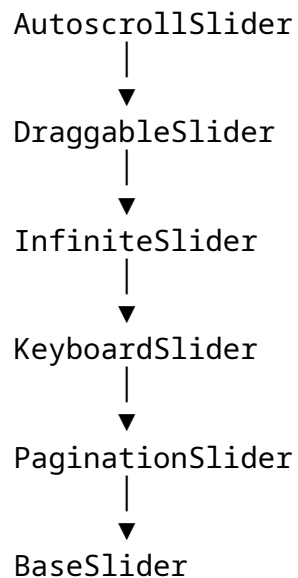
This lets `EventManager` perform the subscription work.

6. `EventManager` — Static Event Maps

A component can expose an event map on its constructor.

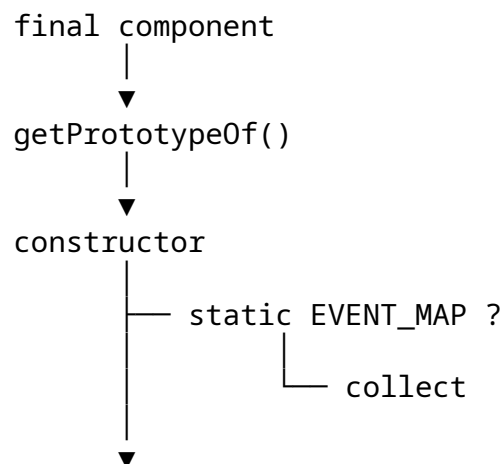
The manager discovers event maps by walking the prototype chain.

For the final slider:



The manager checks every constructor for the configured event-map key.

Conceptually:



parent prototype
↓
...

When the same event is already registered, the manager does not blindly replace the existing handler.

This lets the prototype chain contribute event declarations without turning the final class into one giant manually maintained map.

7. EventManager — Dynamic Subscriptions

Static events are only half of the problem.

Dragging creates a temporary interaction mode.

```
normal mode
  |
  | pointer down
  |
  ▼
drag mode
  |
  | mousemove
  | touchmove
  | mouseup
  | touchend
  |
  | pointer released
  |
  ▼
normal mode
```

`EventManager.subscribe()` and `unsubscribe()` are used for these temporary maps. This gives the component a clean lifecycle:

```
_startDragging()
  |
  |
  ▼
subscribe dynamic events

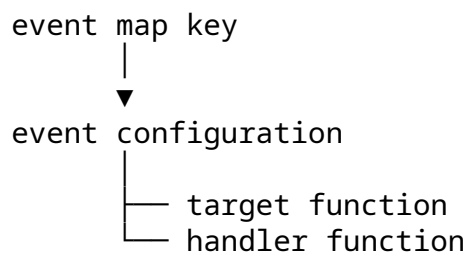
_stopDragging()
  |
  |
  ▼
unsubscribe dynamic events
```

The infrastructure therefore owns the repetitive DOM API work.

8. EventManager Contract Validation

The event manager does not assume a configuration entry is correct.

It validates:



The target function is evaluated against the current instance:

```
const targetElement = target(instance);
```

The resulting DOM element is validated before subscription.

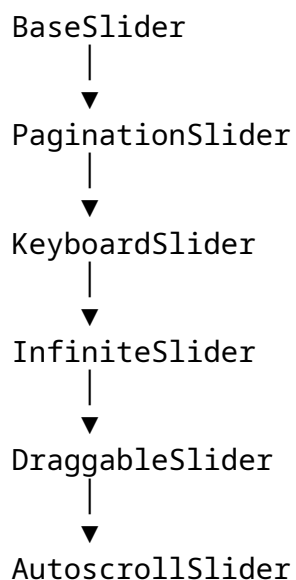
The conceptual contract is:

```
valid map
+ valid target
+ valid handler
    ↓
valid subscription
```

A broken map should surface as a configuration/contract problem rather than a mysterious later failure.

9. Slider Prototype Hierarchy

The complete slider hierarchy is:



The design is intentionally cumulative.

Base

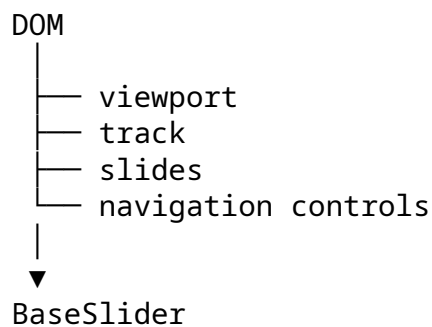
- + pagination
- + keyboard
- + infinite
- + dragging
- + autoscroll
- = final slider

The subclasses extend an existing behavioral contract instead of reimplementing the previous layer.

10. BaseSlider — Core Responsibilities

BaseSlider owns the foundational slider mechanics.

Conceptually:



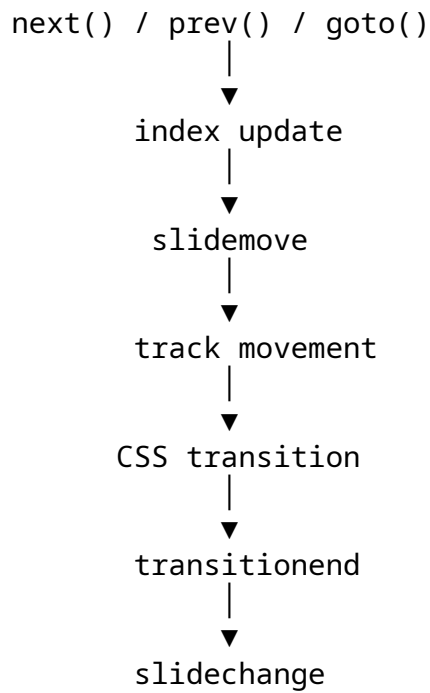
It is responsible for things such as:

- DOM discovery;
- slide collection;
- active index;
- track positioning;
- previous/next navigation;
- transition state;
- resize handling;
- normalization contract;
- core state.

The important architectural point is that later subclasses can use these mechanisms without knowing the DOM mechanics again.

11. BaseSlider — Movement Lifecycle

A logical navigation operation follows this pattern:



The distinction between movement-start and committed slide change is important.
slidemove means:

a slider movement has begun

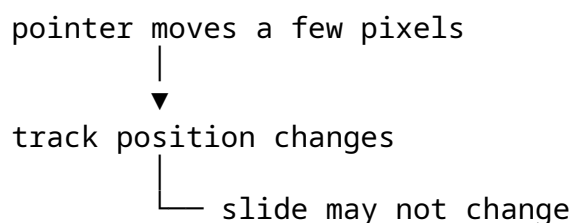
slidechange means:

the active slide has changed

This separation prevents external logic from treating every small or transient movement as a committed slide change.

12. Why This Distinction Matters

Consider drag input.



The architecture therefore should not equate:

pointer/track movement
=
slide change

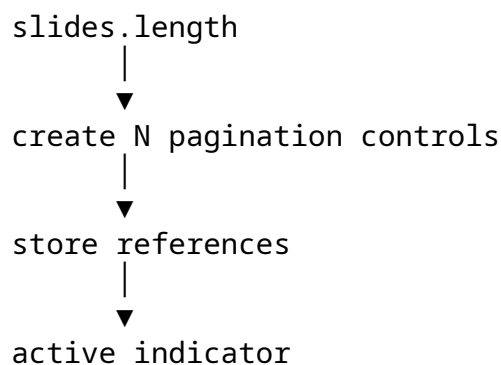
The finalized slide event belongs later in the lifecycle.

This distinction was especially important when reproducing a subtle issue where an interaction generated an event even though the visible slide had not actually changed.

13. PaginationSlider

Pagination is data-dependent.

Navigation controls such as previous/next can exist independently of the data set, while pagination indicators depend directly on slide count.



The subclass therefore owns pagination-specific creation and synchronization.

The architecture does not force every static UI control to be dynamically generated merely because pagination is.

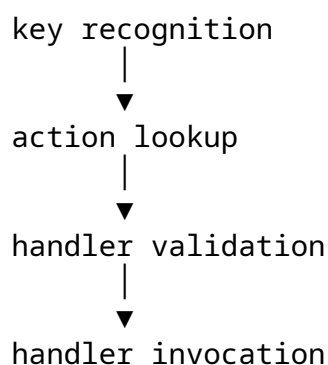
The useful principle is:

```
fixed UI → markup can own it
data-dependent UI → code can generate it
```

14. KeyboardSlider

KeyboardSlider connects slider semantics to the reusable KeyboardManager.

The manager handles:



The component handles:

What should "next" mean to a slider?
What should "reset" mean?
What should "execute" mean?

This is a clean separation between:

input mechanism

and:

domain operation

15. Keyboard Configuration as a Strategy

A keyboard map can conceptually look like:

```
next  → ArrowRight / D
prev  → ArrowLeft  / A
...
```

The manager does not need a different algorithm for every component.

It receives configuration and derives the corresponding action.

That is Strategy-like behavior selection:

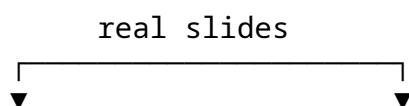
```
same manager
├── slider config
├── audio config
└── app config
```

The mechanism stays stable while the selected actions vary.

16. InfiniteSlider

InfiniteSlider changes the indexing model while preserving the public navigation API.

The visible track becomes conceptually:

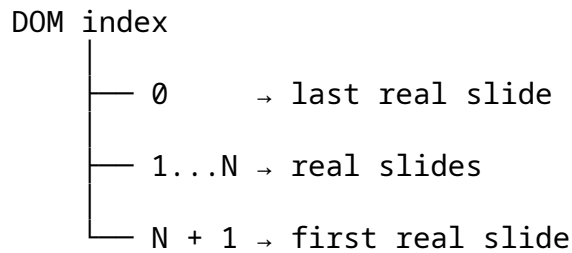


[last clone] [1] [2] ... [N] [first clone]

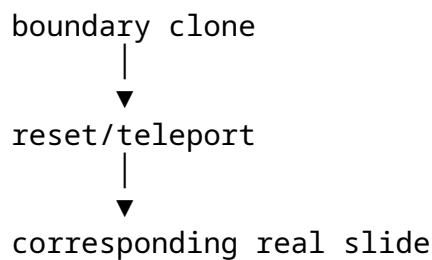
The clone positions exist only to make the transition visually continuous.

17. Infinite Loop Teleport

The logical index and DOM index are therefore not always identical.



When a boundary clone is reached:

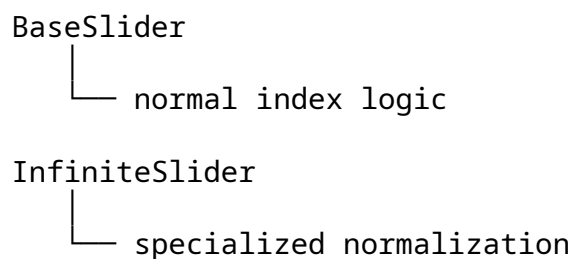


The user sees continuous motion.

The component internally corrects the position without exposing clone mechanics to the rest of the application.

18. Index Normalization

The inheritance hierarchy is useful because the base API can remain stable while the child changes the meaning of an index.



A caller can still use:

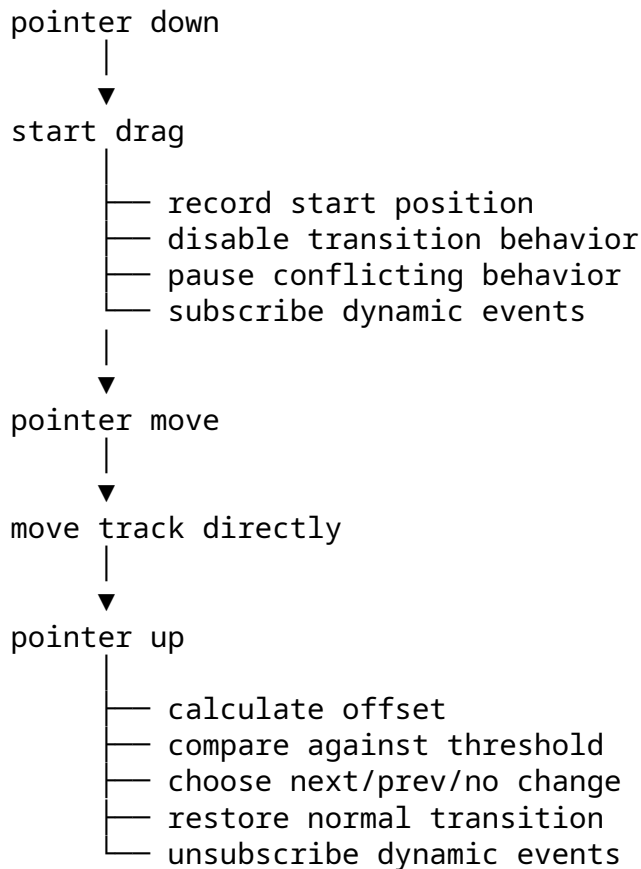
```
next()  
prev()  
goto(index)
```

without knowing whether the slider uses clones.

This is a concrete example of polymorphism reducing external coupling.

19. DraggableSlider — Lifecycle

Dragging has a distinct interaction lifecycle.



The slide changes only when the final offset meets the configured trigger threshold.

20. Drag Threshold

The threshold is not a hardcoded pixel constant.

Conceptually:

```
threshold =  
    current slide width  
    × configured coefficient
```

This makes the interaction relative to the current geometry.

That is especially useful after resize.

21. AutoscrollSlider

Autoscroll is layered on top of the draggable slider.

Its responsibilities include:

timer lifecycle
autoscroll state
pause conditions
lock/unlock
wake-up timing
automatic transition marker

The timer does not know that it is driving a slider.

It simply invokes a callback.

This keeps:

Timer

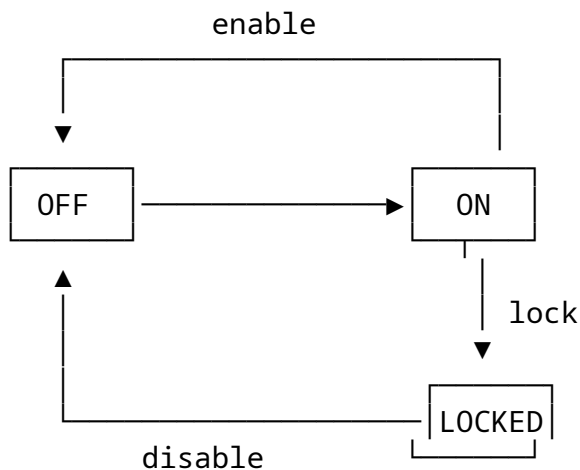
generic and:

AutoscrollSlider

domain-specific.

22. Autoscroll State Machine

The three-state model is:



But there is an important semantic distinction:

OFF

means autoscroll is not active.

LOCKED

means it is temporarily prevented from operating by another part of the application.

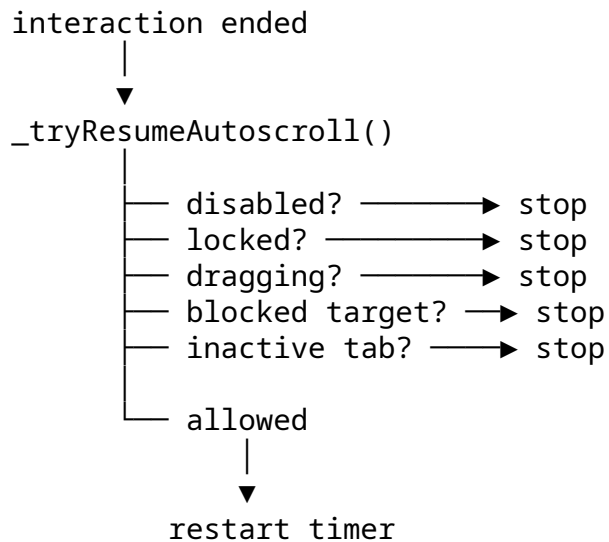
That is why album playback can lock autoscroll without redefining its own domain.

23. Autoscroll Resume Logic

Autoscroll does not simply restart whenever an interaction ends.

It checks whether resumption is actually allowed.

Conceptually:



This centralization is important because several independent conditions can prevent resumption.

Without it, each interaction handler would have to know about all the others.

24. Autoscroll Wake-Up Timing

The implementation distinguishes between:

normal autoscroll interval

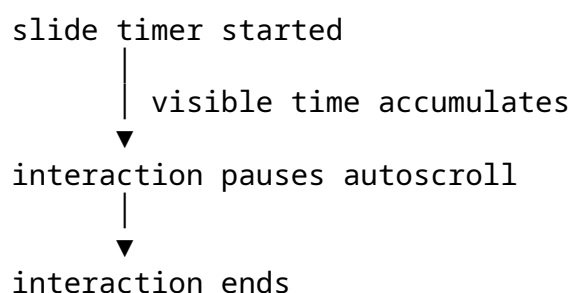
and:

wake-up/restart delay

The purpose is not just to delay the next movement arbitrarily.

The timing can be calculated in relation to how long the current slide has already been visible.

Conceptually:



↓
remaining / wake-up interval
↓
normal autoscroll resumes

This preserves a more natural overall slide duration.

25. Automatic vs Manual Movement

Autoscroll stores an internal marker for operations it initiated.

`_isAutoscrollAction`

The reason is subtle:

automatic movement

should not accidentally trigger logic intended only for:

manual interaction

The marker therefore acts as a contextual signal inside the slider's control flow.

26. AudioPlayer — Domain Boundary

AudioPlayer is a separate stateful component.

It owns:

HTMLAudioElement interaction
album selection
track selection
theme playback
playlist progression
audio state
media-session interaction

The application does not directly manipulate the native audio object.

Instead it calls semantic commands:

`play()`
`pause()`
`toggle()`

`playTheme()`

```
resetTheme()  
  
nextAudioTrack()  
prevAudioTrack()  
switchAudioTrack()  
  
nextAlbum()  
prevAlbum()  
switchAlbum()  
  
restart...  
rewind...  
stop...
```

This is much more meaningful than exposing:

```
audio.src = ...  
audio.currentTime = ...
```

throughout the application.

27. Audio FSM

The `AudioPlayer` maintains four application-level modes:

```
IDLE  
THEME  
ALBUM  
ALBUMTHEME
```

The important idea is that the native `<audio>` state is not enough to represent application meaning.

For example:

```
audio.paused
```

does not tell the application whether the currently managed context is:

```
main theme
```

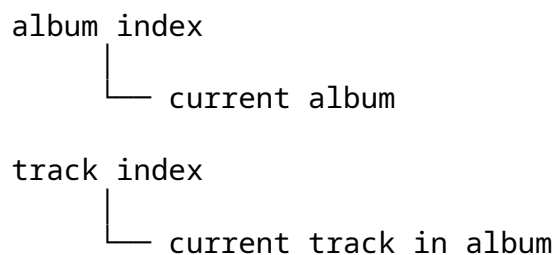
or:

```
album playback
```

The FSM supplies that higher-level semantic information.

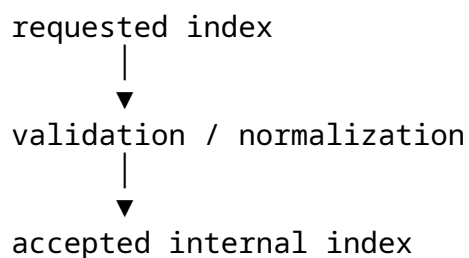
28. Audio Track and Album Indices

The player maintains separate positions.



The track queue setter validates and normalizes the requested track index before accepting it.

Conceptually:



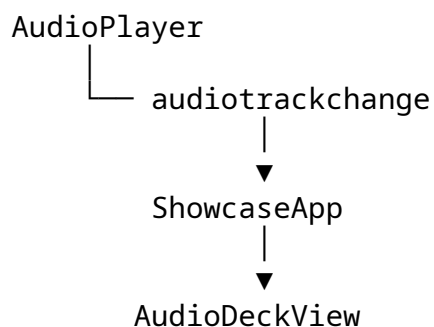
This prevents invalid queue positions from propagating into playback logic.

29. Audio Event Model

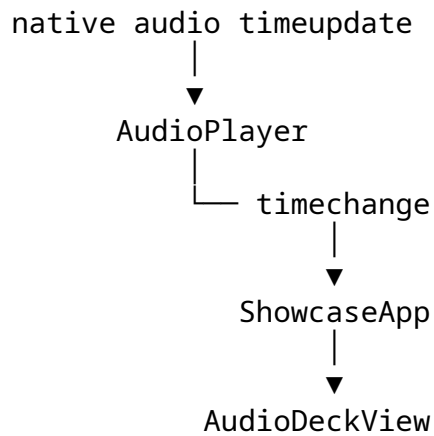
AudioPlayer communicates through custom events.

albumplay
albumpause
albumplaypassthrough
albumend
audiotrackchange
timechange

A typical flow is:



For timeline updates:

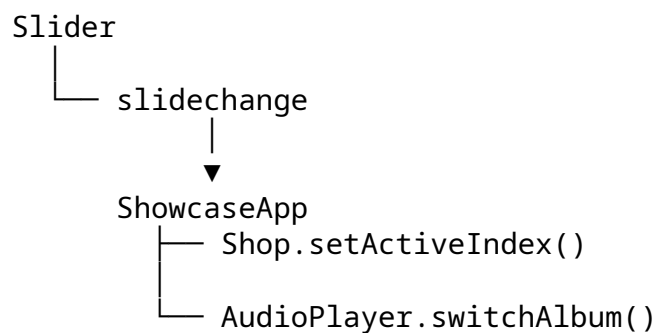


The view therefore remains passive with respect to playback.

30. Slider <-> Audio Synchronization

The synchronization rule is application-level.

When the active slide is committed:

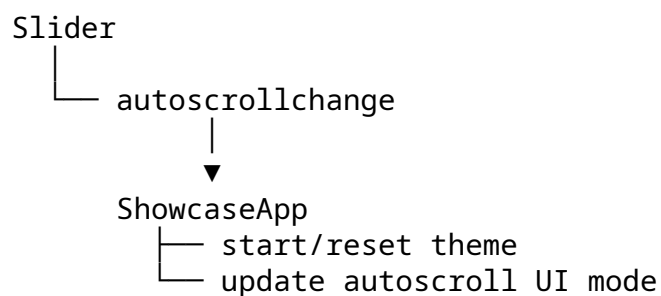


The implementation also checks whether the tab/document is active before switching album content in the relevant flow.

This is another example of ShowcaseApp coordinating consequences rather than components knowing about each other.

31. Autoscroll <-> Audio Synchronization

When autoscroll changes state:



So the application maps:

autoscroll ON

to:

main theme active

and:

autoscroll OFF

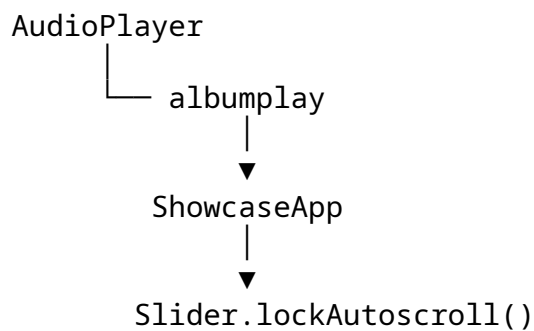
to:

theme reset

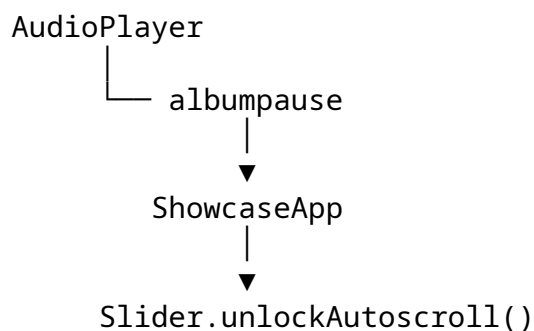
This is intentionally outside the Slider itself because the Slider should not know that an `AudioPlayer` exists.

32. Album Playback Locks Autoscroll

The inverse relationship is also application-level.



When the album pauses:



Again:

`AudioPlayer` knows `audio`
`ShowcaseApp` knows the application rule
`Slider` knows `autoscroll`

No component has to know the complete architecture.

33. albumplaypassthrough

There is a special application-level case where album playback input can be treated as a passthrough condition.

The resulting coordination can disable autoscroll:

```
special audio interaction
    |
    ▼
albumplaypassthrough
    |
    ▼
ShowcaseApp
    |
    ▼
Slider.disableAutoscroll()
```

This is another example of why the application coordinator exists: it can translate one subsystem's event into another subsystem's command without creating a direct dependency between them.

34. Album End

When album playback reaches the end:

```
AudioPlayer
    |
    └─ albumend
        |
        ▼
    ShowcaseApp
```

The application then advances the slider.

When the document is active, the normal animated navigation path can be used.

When it is not active, the implementation can use:

```
nextInstantly()
```

instead.

This prevents background-tab behavior from forcing an unnecessary visual transition.

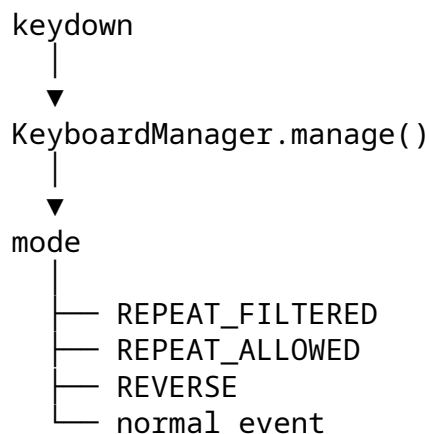
35. Input Modes in ShowcaseApp

Keyboard handling includes more than key-name lookup.

ShowcaseApp classifies special conditions such as:

repeat event
passthrough condition
modifier state
reverse routing
execute mode
toggle mode

A simplified representation is:



This application-level layer is what lets the manager remain generic.

36. Repeat Filtering

The system distinguishes:

ordinary keydown

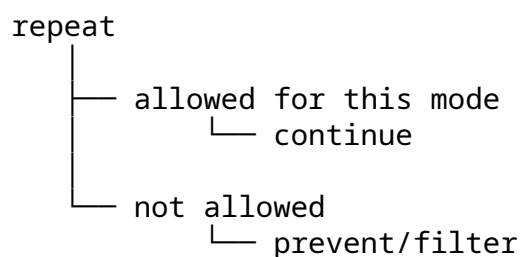
from:

keydown with e.repeat

But not every repeat should be rejected.

There are explicit exceptions for actions where repetition is meaningful.

Therefore:



The decision belongs to ShowcaseApp, because it depends on application-wide routing and the state of another component.

37. Reverse Pipeline

A special audio passthrough case can reverse the normal component order.

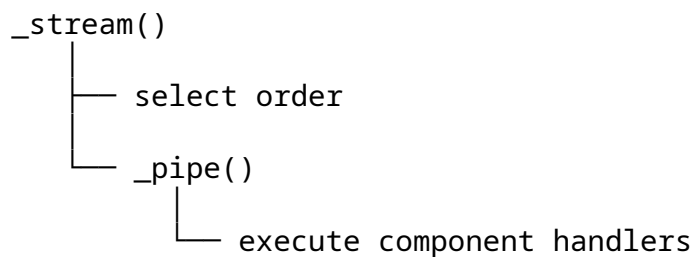
Normal:

Slider → AudioPlayer → Shop

Reverse:

AudioPlayer → Slider → Shop

The important implementation detail is that `_stream()` chooses the pipeline and then delegates the actual loop to `_pipe()`.

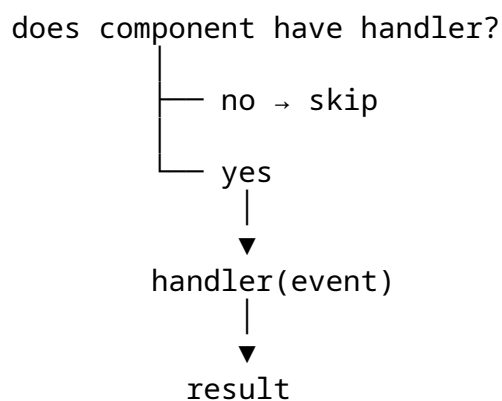


This is a clean example of a reusable pipeline primitive rather than duplicated conditional code for every input type.

38. `_pipe()` Contract

The pipeline iterates over components.

For each one:



The event reference is updated with the returned value.

This gives the chain its control semantics:

true
→ consumed

original Event

→ continue

other result

→ application-specific semantic value

The subtle point is that the event itself becomes a continuation token.

It is not just input data; it can also mean:

I did not consume this.

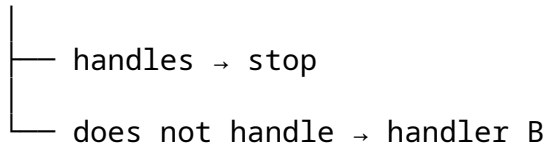
Keep going.

39. Why This Resembles Chain of Responsibility

The structure is not a textbook GoF class hierarchy named Handler.

Still, the behavioral principle is the same:

handler A



The project combines that with:

inheritance

because child and parent handlers can cooperate.

Therefore the best description is:

A lightweight Chain of Responsibility protocol implemented through pipelines and polymorphic handlers.

40. Inheritance + Parent Handler Cooperation

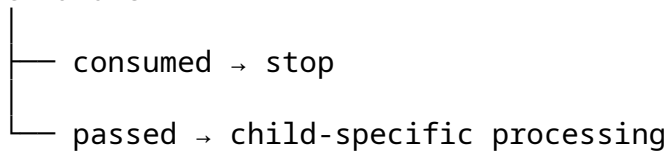
A specialized handler can invoke its parent logic before adding its own behavior.

Conceptually:

Child handler



Parent handler



This matters because each layer can preserve what the previous layer already implemented.
The result is:

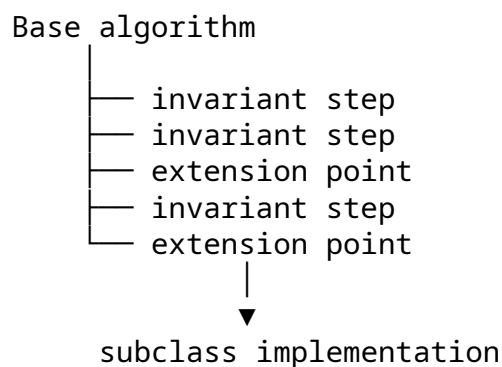
```
Base behavior
+
pagination behavior
+
keyboard behavior
+
infinite behavior
+
drag behavior
+
autoscroll behavior
```

without copying the entire slider implementation into every subclass.

41. Template Method-Like Structure

The slider hierarchy contains Template Method-like structures where a base algorithm performs invariant steps and subclasses customize specific phases.

Conceptually:



This classification is appropriate only where the base implementation actually establishes an algorithm and subclasses specialize an extension point.

It is not applied to every overridden method automatically.

42. Static Class Contracts

The architecture also makes use of static class-level contracts.

Examples include state tables and component-specific configuration constants.

The important distinction is between:

fixed implementation constant

and:

polymorphic static contract

43. Module Constants vs Static Properties

A module-level constant is appropriate when the concept is internal and fixed:

```
STATES
KEYBOARD_MODES
mouse button constants
```

A static property/getter is useful when the value forms part of the component contract.
For example:

```
Class.STATES
Class.EVENT_MAP_KEY
Class.DEFAULT_URL
```

can expose a named class-level contract.

The architecture intentionally does not turn every constant into a static property.

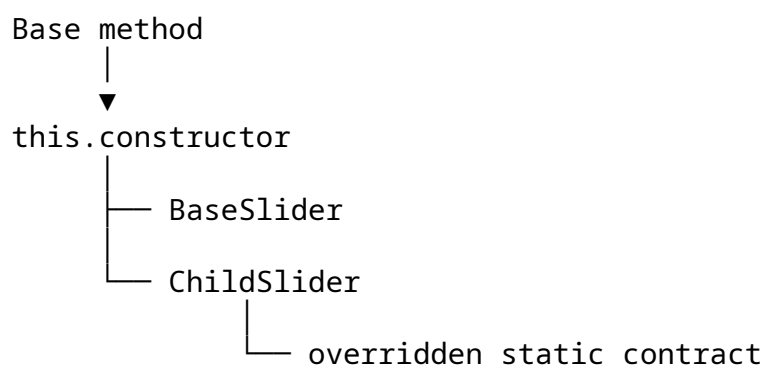
44. `this.constructor` and Polymorphic Static Lookup

When a static contract is intentionally overridable, inherited methods can use:

```
this.constructor.STATES
```

rather than pinning the lookup to the base constructor.

Conceptually:



This is useful in state setters and similar extension points.

The important rule is:

```
use this.constructor
when runtime-class polymorphism is intended
```

not:

use `this.constructor` everywhere “just in case”

For a concrete implementation-specific contract, an explicit class reference can be clearer.

45. Read-Only Static Contracts

Where a static contract is exposed read-only, the implementation can use getter-based access or an immutable value.

The desired semantics are:

`consumer` can read
`consumer` cannot replace

and, where appropriate:

`consumer` cannot mutate contents

The specific mechanism is less important than the contract itself.

46. Shop and Default URL

Shop is a small example of a class-level default contract.

The default URL is intentionally read-only and may be resolved through the actual constructor when polymorphic static behavior is desired.

The architecture therefore separates:

instance-specific shop URL

from:

class-level fallback URL

This keeps the default policy from being duplicated throughout the component.

47. ButtonManager and KeyboardManager Return Contract

The project intentionally standardized both managers around the same result behavior.

Conceptually:

```

action returns value
├── true
│   └── handled
└── null/undefined
    └── preserve original event

```

This means a manager can be used inside a pipeline without inventing a second incompatible protocol.

The consistency itself is architectural value.

48. Mouse and Button Pipeline

Click events can be streamed through the same pipeline model.

```

MouseEvent
  ↓
ShowcaseApp
  ↓
Slider.handleClick()
  ↓
AudioPlayer.handleClick()
  ↓
Shop.handleClick()

```

The exact behavior depends on which component actually defines the corresponding handler.

The pipeline infrastructure remains generic.

49. Auxiliary Mouse Input

The application also distinguishes mouse-button semantics.

Examples include:

```

left button
middle button
right button

```

Right-click is used for a temporary button state.

The application stores:

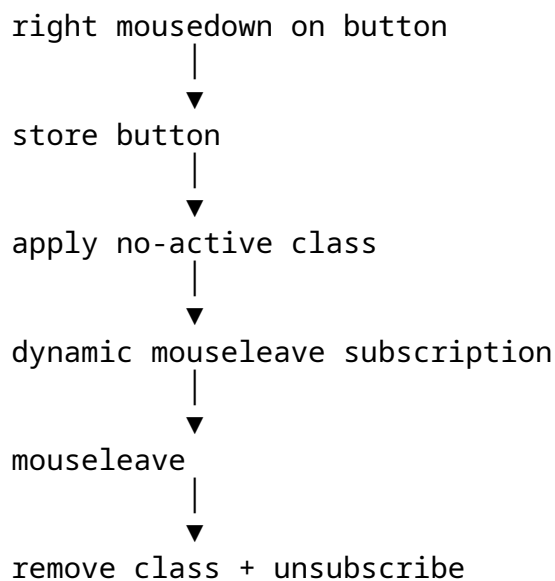
```

_btnNoActive

```

and dynamically subscribes to the button's mouseleave.

The lifecycle is:

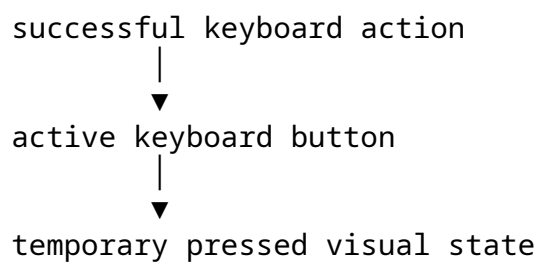


This is another concrete use of dynamic EventManager subscriptions.

50. Focus and Keyboard UI State

ShowcaseApp manages some application-wide keyboard visual state.

For example:



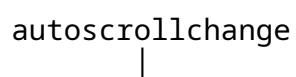
A matching keyup removes the state.

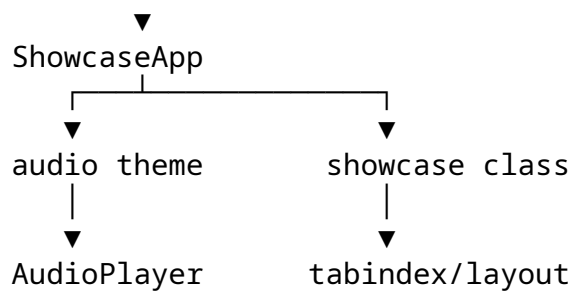
The component that understands the semantic keyboard action does not have to implement the document-level focus cleanup itself.

51. Autoscroll and UI Mode

Autoscroll changes more than a timer.

ShowcaseApp maps the autoscroll state to UI mode:



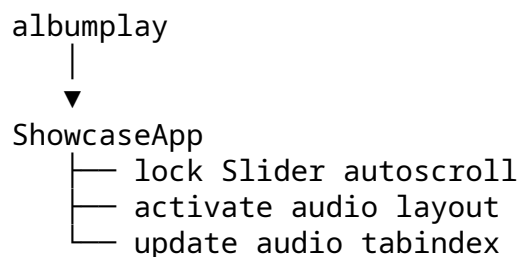


So a state change in one component can have several application-level visual/audio consequences.

This is exactly the type of coordination that belongs in the application layer.

52. Audio Mode and UI Mode

Album playback similarly activates an application-level audio UI mode.



When playback pauses, the opposite operations can occur.

This keeps:

AudioPlayer

focused on audio while:

ShowcaseApp

knows how audio affects the broader showcase UI.

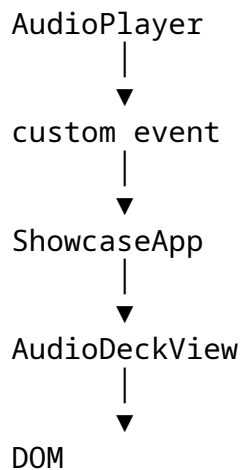
53. AudioDeckView

AudioDeckView has a deliberately narrow role.

It renders information such as:

current track
 total tracks
 track title
 timeline
 current time
 duration

Its architecture is:



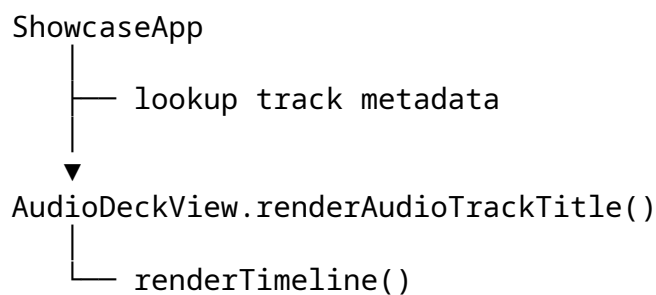
The view does not decide when tracks change.
That decision belongs to AudioPlayer.

54. Audio Track Change

The event carries enough information for the application to update the view.
Conceptually:

```
trackIndex  
albumIndex  
totalTracks
```

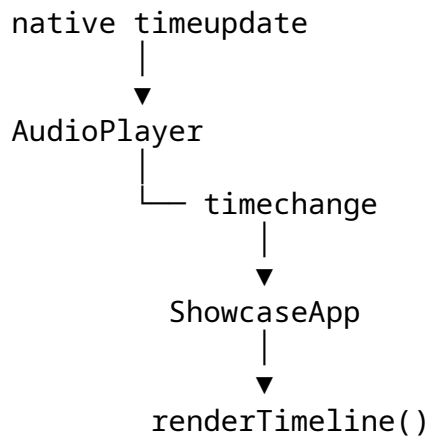
Then:



The view therefore stays unaware of the playlist structure.

55. Audio Timeline

The native audio element emits time information.
The AudioPlayer converts that into a project-level event.

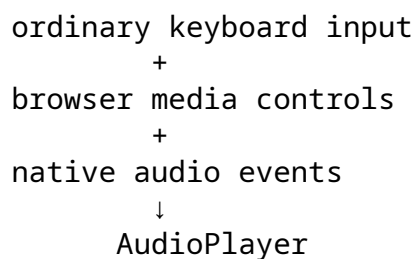


This keeps browser-specific media events inside the audio domain.

56. Media Session

The player also integrates Media Session behavior because browser/media shortcuts can bypass ordinary keyboard handling.

This is important architecturally:



All of these external mechanisms are translated into the same semantic playback API.

The rest of the application does not need to know which input source caused:

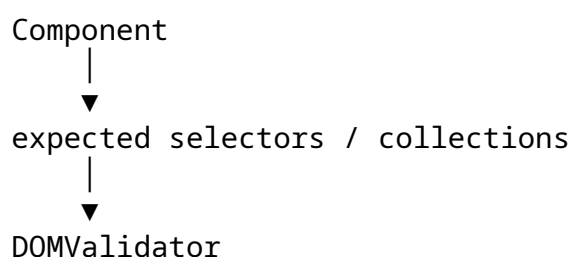
```

next track
pause
play
previous track
  
```

57. DOM Validation as Explicit Contract

Every major component relies on a known DOM structure.

The design treats that structure as an API contract.



A missing required dependency is an initialization error.

The architectural philosophy is:

```
explicit contract
>
silent recovery
```

This is particularly useful in an educational architecture project because a broken integration becomes visible immediately.

58. Why Fallback Selector Guessing Is Avoided

Suppose the component expects:

```
.slider__track
```

and it is missing.

A fallback implementation could search several unrelated selectors.

That may keep the page running temporarily, but it makes the actual contract unclear.

This project instead prefers:

```
missing expected DOM
  |
  ▼
fail near initialization
```

That is a deliberate fail-fast choice.

59. Resize Lifecycle

Slider geometry depends on the actual viewport/track dimensions.

The architecture therefore recalculates position when dimensions change.

Conceptually:

```
resize
  |
  ▼
measure current geometry
  |
  ▼
recalculate slide offset
  |
  ▼
move track to current logical position
```

This is especially important for:

drag threshold

and:

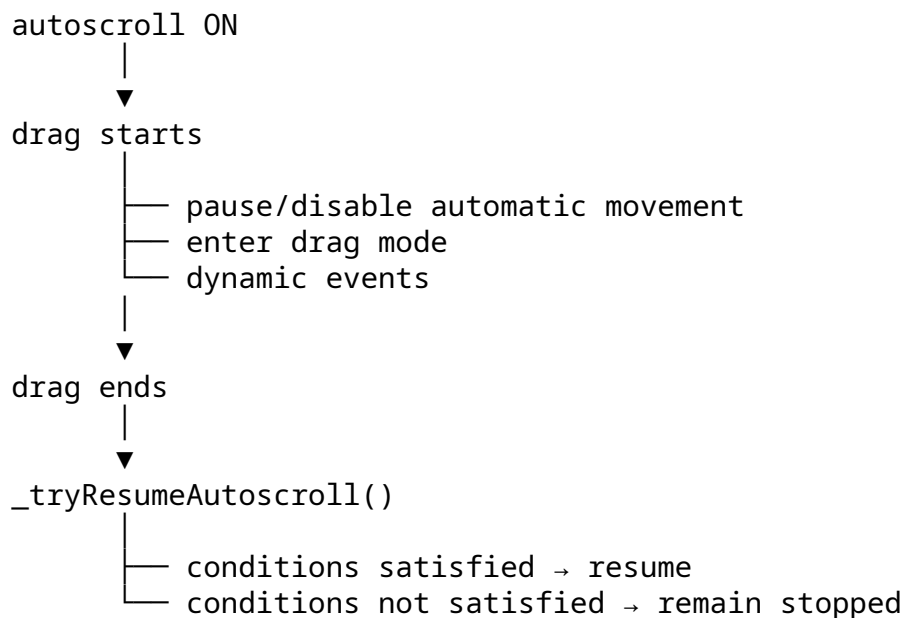
infinite-loop positioning

because both depend on current slide geometry.

60. Dragging and Autoscroll Interaction

Dragging and autoscroll compete for control of the same track.

The intended lifecycle is:



This illustrates why autoscroll state cannot be reduced to a single boolean.

61. Audio and Autoscroll Are Separate FSMs

An important architectural property is that the following can exist simultaneously:

Slider state	= IDLE
Autoscroll state	= LOCKED
Audio state	= ALBUM

These are not contradictory because they describe different concerns.

This is exactly why a single global FSM would be awkward.

62. Configuration Structure

The project stores UI and interaction rules in structured configuration.

Typical categories include:

```
singleSelectors
groupSelectors
classes
classesActive
jsClasses
states
click
press
autoplay
autoscrollDelay
autoscrollWakeUpDelay
slideTriggerThresholdCoef
```

This allows one implementation to be driven by a specific configuration without embedding every selector/key inside the component code.

63. Configuration as a Contract

The configuration is not just a bag of options.

Parts of it form contracts:

```
action name
  |
  ▼
expected handler
  |
  ▼
component method exists?
```

and:

```
event definition
  |
  └─ target function
  └─ handler function
```

The managers validate those relationships.

The result is a small declarative contract system inside the application.

64. Micro-Framework Characteristics

Some infrastructure has started to resemble a small internal framework:

EventManager
KeyboardManager
ButtonManager
DOMValidator
Timer

The important point is not to call it a framework for marketing purposes.

The useful observation is:

The application has extracted repeated architectural mechanisms into configurable reusable services.

That is why the same ideas can be applied to several components.

65. What Is Intentionally Not Abstracted

Not every repeated idea has been promoted into another abstraction.

This is important.

For example:

fixed state table
fixed implementation-specific constant
single-use local calculation

does not automatically justify another manager or generic service.

The project deliberately favors abstractions where they represent a recurring architectural mechanism.

That is the difference between:

useful abstraction

and:

abstraction for abstraction's sake

66. Why the State Pattern Is Not Forced

The project already has explicit state transitions.

For example:

this._state

plus:

state constants
state setter
transition logic

That is enough to model a finite state machine.

A separate State object hierarchy would only become valuable if:

state-specific behavior grows substantially

and transitions become difficult to keep inside the owning component.

At the current scale, local FSMs are clearer.

67. Pattern Map — More Precise Classification

GoF / GoF-like

Template Method-like → slider inheritance
Mediator-like → ShowcaseApp
Chain of Responsibility
→ input pipeline
Strategy-like → configurable managers
Command-like → Button

ARCHITECTURAL / OTHER

Dependency Injection → dependencies supplied externally
Inversion of Control → composition outside components
Composition → app assembled from parts
FSM → state modeling
Pub/Sub / Observer-like
→ EventManager + custom events
Fail-fast validation → DOMValidator
Polymorphism → subclass behavior/static contracts

This distinction matters because not every useful architecture concept is a GoF design pattern.

68. Static Inheritance in the Prototype Version

The prototype implementation establishes both forms of inheritance:

Instance inheritance
Child.prototype
↓
Parent.prototype

and:

Static inheritance
Child
|
▼
Parent

This allows static class contracts to participate in inheritance as well.
Conceptually:

KeyboardSlider.STATES
|
└─ may resolve through static inheritance
 |
 ▼
 PaginationSlider.STATES

This is why using:

`this.constructor.STATES`

can preserve polymorphic behavior.

69. Alias Discipline

The architecture also benefits from avoiding unnecessary local aliases.

Good:

use a descriptive module constant directly

when it is used only once or in a small area.

Good:

create a short alias

when the alias is meaningfully reused many times and improves local readability.

The principle is not “never alias”.

It is:

alias only when it buys clarity or reduces repeated noise

This keeps the implementation readable while preserving the larger architecture.

70. Prototype Version vs Class Version

The two versions represent the same conceptual architecture.

```
architecture
├── prototype syntax
└── class syntax
```

The prototype version exposes more of the mechanics explicitly:

```
prototype chain
constructor restoration
static inheritance
Object.setPrototypeOf(...)
```

The class version expresses the same ideas through:

```
class
extends
static
super
```

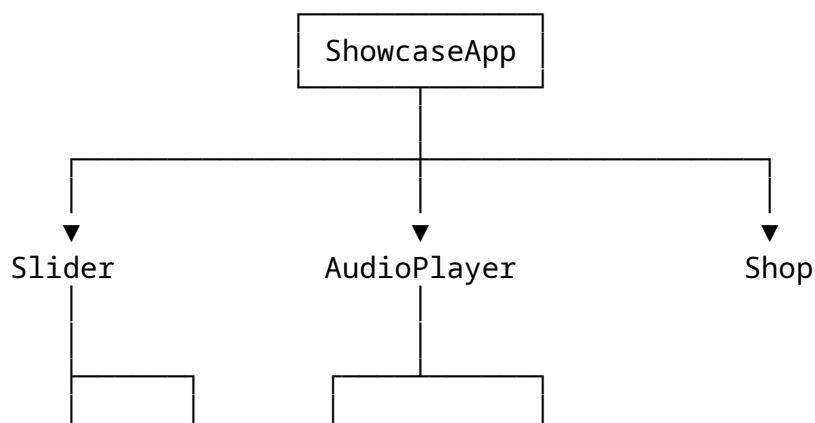
The architectural model itself remains:

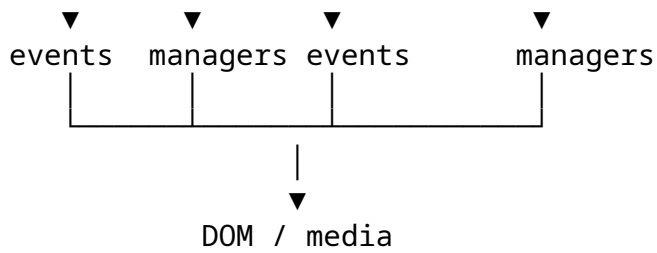
```
same component boundaries
same FSMs
same managers
same event pipelines
same application coordination
```

That is why this documentation can be shared between both projects.

71. The Architecture as a Small Internal Ecosystem

At runtime, the application behaves like a small ecosystem of cooperating objects.

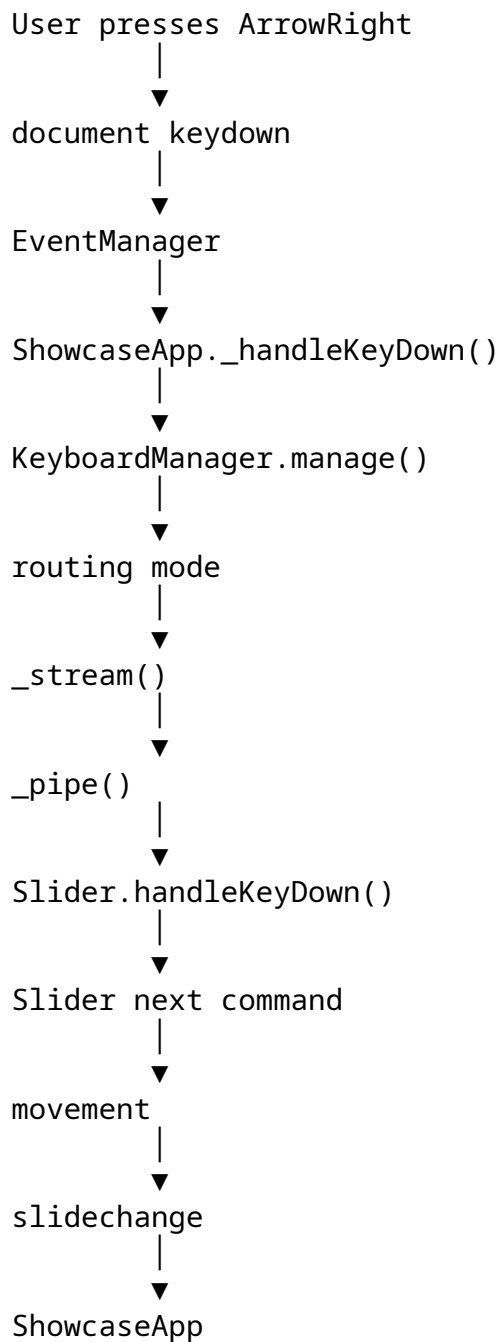




The system is complex, but each object has a recognizable reason to exist.

72. Typical User Interaction — Keyboard

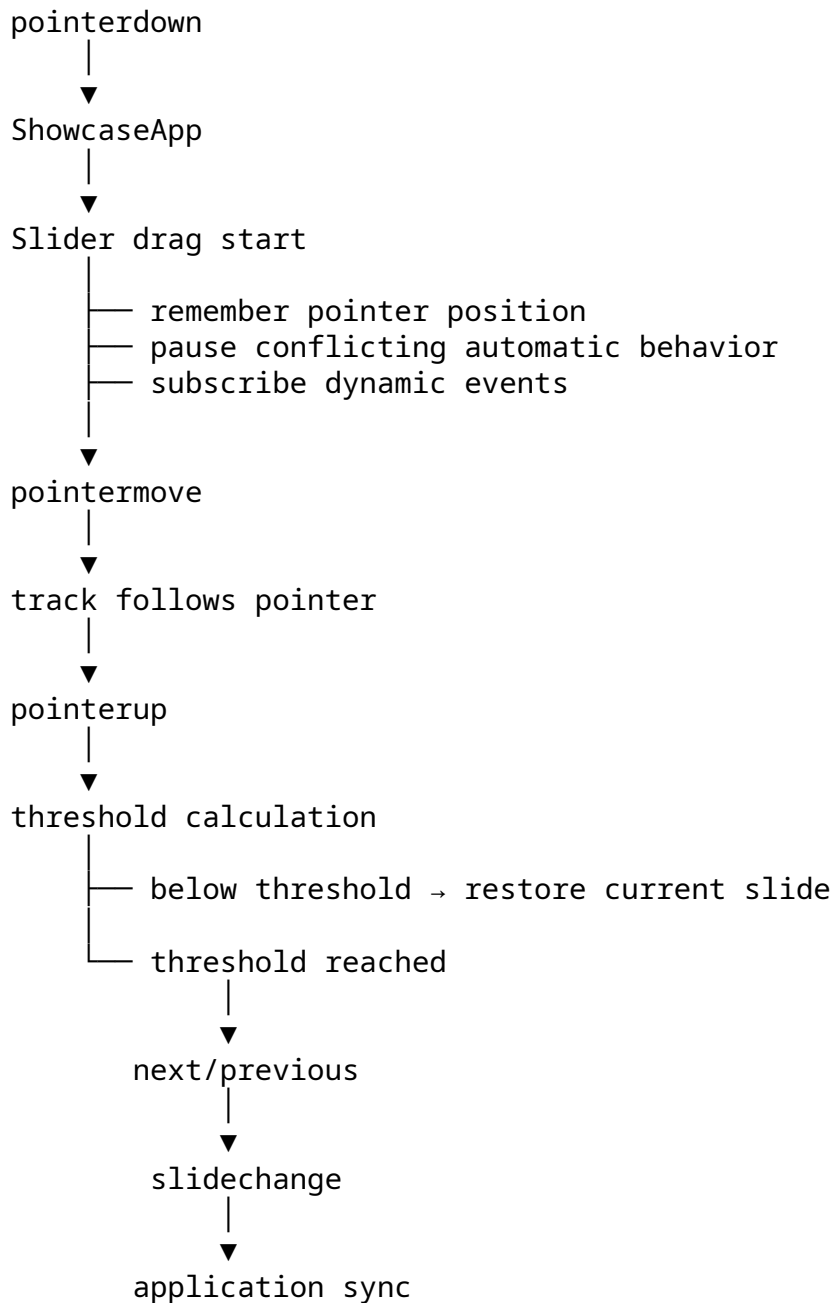
Let's follow one complete interaction.



└─ Shop
└─ AudioPlayer

One browser event therefore passes through several architectural layers.

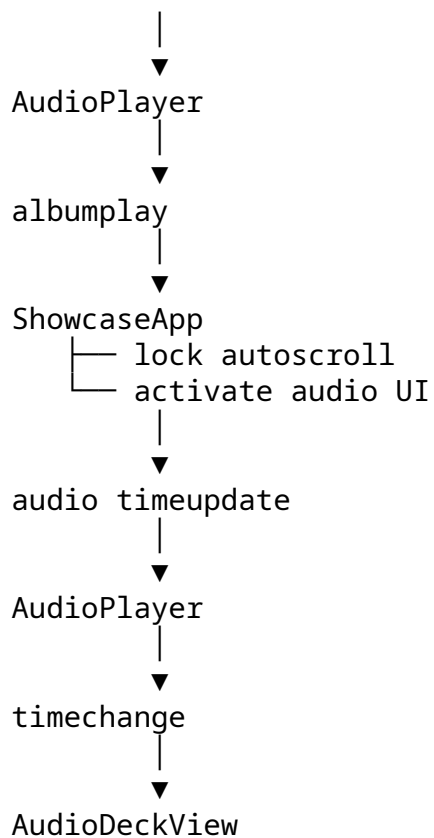
73. Typical User Interaction — Drag



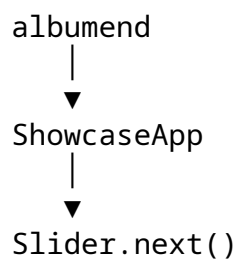
A drag is therefore a temporary interaction mode, not just another click handler.

74. Typical User Interaction — Album Playback

User starts album

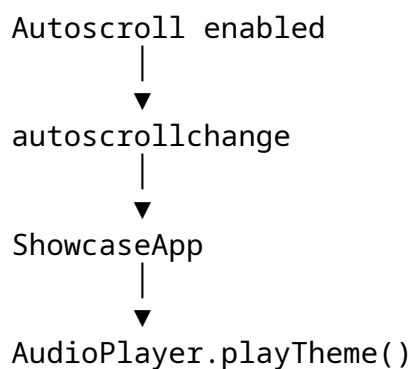


When the album finishes:



The audio system therefore influences navigation without directly depending on Slider internals.

75. Main Theme / Autoscroll Interaction



When autoscroll is turned off:

```

autoscrollchange
  ↓
ShowcaseApp
  ↓
MediaPlayer.resetTheme()

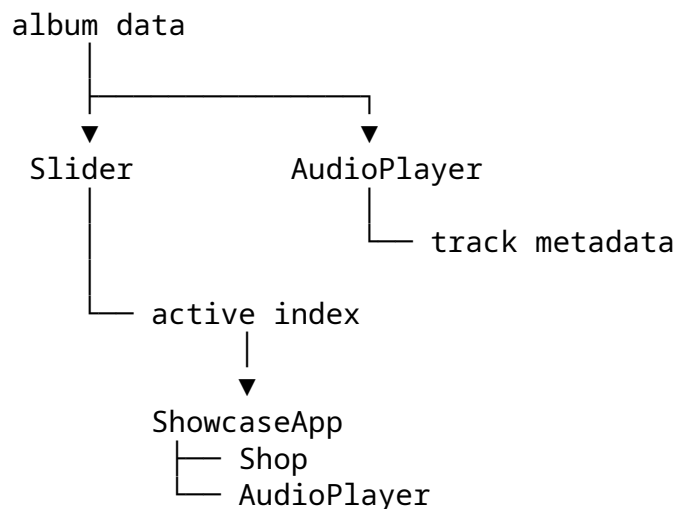
```

This is a clean example of application-level policy.

76. Data Flow

Album information is shared through configuration/data rather than hidden duplicated structures.

Conceptually:



The current slide index acts as the synchronizing identity between the visual showcase and the album data.

77. Component Contracts

The public surface of a component should be semantic.

Examples:

```

Slider
  |--- next()
  |--- prev()
  |--- goto()
  |--- lockAutoscroll()
  |--- unlockAutoscroll()
  |--- ...

```

```
AudioPlayer
├── play()
├── pause()
├── toggle()
├── playTheme()
├── switchAlbum()
├── switchAudioTrack()
└── ...
```

```
Shop
├── setActiveIndex()
```

The application should prefer:

`semantic command`

over:

`reach inside the component and mutate internals`

This is one of the main forms of encapsulation in the design.

78. Error Boundaries and Validation

The project validates several categories of assumptions:

DOM structure
event map structure
handler existence
target element availability
state tokens
index values
configuration values

This produces a general philosophy:

`invalid contract`

↓



`detect close to source`

rather than:

`invalid contract`

↓



`allow inconsistent state`

↓



`crash somewhere unrelated later`

79. Why the Architecture Is More Complex Than a Typical Slider

A minimal implementation could use:

- one object
- one event handler
- one interval
- one audio element

This project instead introduces:

- multiple component boundaries
- multiple managers
- multiple state machines
- multiple event layers
- inheritance
- polymorphism
- configuration
- validation
- dynamic subscriptions

That complexity is deliberate.

The project is a learning laboratory for architecture.

The question is therefore not:

"Could this be shorter?"

but:

"Does each abstraction represent a real architectural concern?"

80. Architectural Trade-off

There is a real trade-off.

more abstraction

- better separation
- reuse
- extensibility
- explicit contracts
- more code / mental overhead

The project deliberately accepts this cost because the exercise is about learning architecture.

At the same time, the implementation avoids introducing abstractions that do not have a meaningful job.

That is why there is no:

global State object
generic mediator class for every event
extra wrapper for every constant

unless the architecture actually benefits from one.

81. What Makes the Architecture Coherent

The architecture is not strong because there are many classes.
It is coherent because the classes have different responsibilities.

ShowcaseApp
→ coordinates

Slider
→ controls slider behavior

AudioPlayer
→ controls audio behavior

AudioDeckView
→ renders audio information

Shop
→ controls external link state

EventManager
→ routes subscriptions

KeyboardManager
→ routes keyboard actions

ButtonManager
→ routes button actions

Timer
→ manages timing

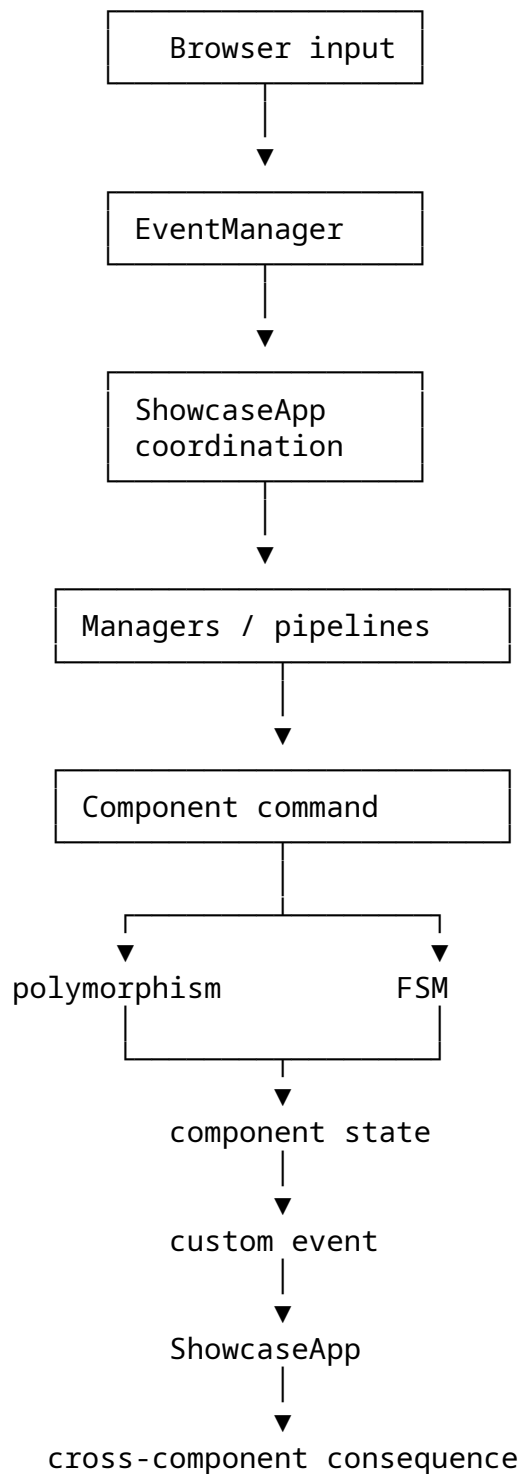
DOMValidator
→ enforces DOM contract

Button
→ represents an executable UI action

The boundaries are meaningful because each object answers a different question.

82. Architectural Core

Everything can be reduced to one central flow:



This loop explains most of the architecture.

83. Final Summary

The system is best understood as a layered architecture:

APPLICATION
└─ ShowcaseApp
 └─ composition + coordination

COMPONENTS

- └─ Slider
- └─ AudioPlayer
- └─ AudioDeckView
- └─ Shop

INFRASTRUCTURE

- └─ EventManager
- └─ KeyboardManager
- └─ ButtonManager
- └─ Timer

VALIDATION / UI ABSTRACTIONS

- └─ DOMValidator
- └─ Button

INTERNAL MECHANISMS

- └─ inheritance
- └─ polymorphism
- └─ FSMs
- └─ pipelines
- └─ dynamic subscriptions
- └─ explicit contracts

The three FSMs are intentionally local:

Slider

- └─ IDLE / MOVING

AutoscrollSlider

- └─ OFF / ON / LOCKED

AudioPlayer

- └─ IDLE / ALBUM / THEME / ALBUMTHEME

The main interaction model is:

event



routing



pipeline



polymorphic command



state transition



custom event



application coordination

And the main architectural rule is:

Components own behavior.
Managers own reusable mechanisms.
ShowcaseApp owns composition and coordination.
FSMs stay with the components that own the state.
Patterns are used where the implementation actually benefits from them.

That combination is what makes the project an architecture exercise rather than merely a slider implementation.

84. Cross-Cutting Architectural Story

Why the Pieces Fit Together

The architecture becomes easiest to remember as a sequence of ownership decisions.

1. A browser event appears.
↓
2. Infrastructure catches/routs it.
↓
3. ShowcaseApp decides application-level order.
↓
4. Manager translates raw input into a semantic action.
↓
5. The owning component executes the action.
↓
6. Local FSM/state changes if necessary.
↓
7. The component publishes a meaningful event.
↓
8. ShowcaseApp applies cross-component consequences.

That sequence is the through-line of the project.

Why Managers Are More Than Convenience Wrappers

A manager earns its place when it removes a mechanism that would otherwise be repeated or mixed into domain objects. `KeyboardManager`, for example, owns key matching, configuration interpretation and handler-contract verification. `ButtonManager` does the analogous job for CSS-class-based button actions. `EventManager` owns subscription lifecycle and event-map discovery. `Timer` owns interval lifecycle.

Without managers

```
component
├─ addEventListener()
└─ removeEventListener()
```

With managers

```
component
└─ domain behavior
    |
```

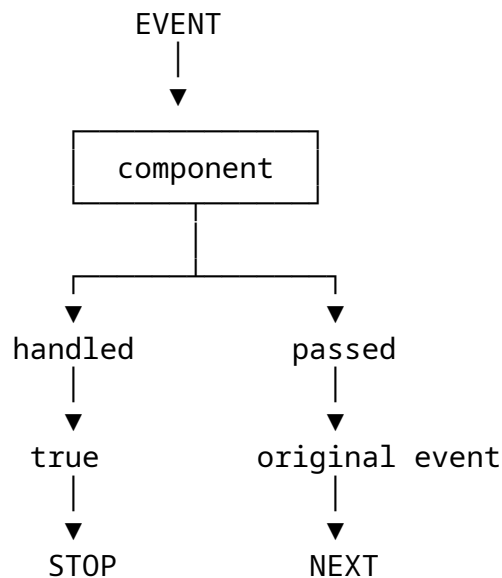
- └ key matching
- └ button lookup
- └ timer bookkeeping
- └ domain behavior

▼
manager handles
reusable mechanism

The point is not fewer lines. The point is that the component's code can speak in domain terms.

Why `_pipe()` Is Worth Documenting

`_pipe()` is one of the places where several architectural ideas meet. It uses composition, polymorphic handlers and an explicit return-value protocol to let one event travel through multiple components without hard-coding all possible combinations.



This is why the pipeline is more than a utility loop. It is a small protocol for cooperation.

Why Local FSMs Are Preferable Here

The project has several kinds of state, but they do not describe the same thing.

Slider FSM	→ movement lifecycle
Autoscroll FSM	→ automatic-motion permission/lifecycle
Audio FSM	→ playback context

Trying to flatten these into one macro-state would create a matrix of combinations rather than make the model clearer. Keeping state local gives each component one authoritative owner.

Why the Architecture Does Not Chase Patterns

A pattern is useful here when it gives a name to a real shape in the code. It becomes harmful when the documentation or implementation starts bending the code merely to justify the pattern.

Therefore the project uses language such as:

Template Method-like

Mediator-like
Command-like
Strategy-like
Observer/Pub-Sub-like

when the correspondence is partial or adapted to native JavaScript. The architecture is the substance; the pattern name is only a useful vocabulary for discussing it.

The Educational Goal

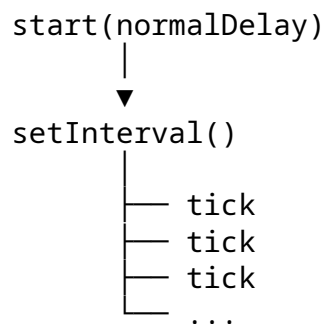
This project was built as an architectural laboratory around a deliberately simple product idea. A slider and an audio showcase are small enough to understand completely, yet rich enough to expose problems involving interaction, state, timing, event propagation, inheritance and cross-component coordination.

That makes the project useful as a study object: the UI gives the architecture something concrete to control, while the architecture gives the UI a laboratory in which OOP and system-design ideas can be explored without hiding them behind a framework.

85. Timer Deep Dive: Why bootstrap Exists

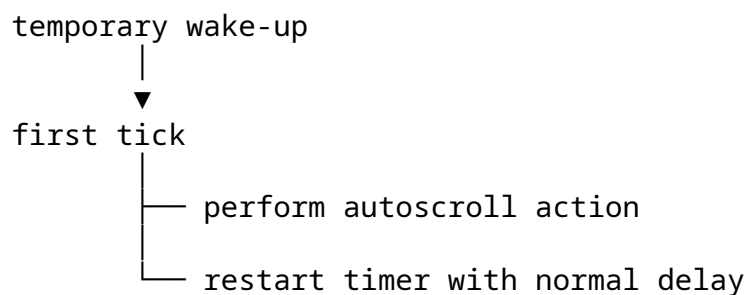
Timer looks small, but one of its details is architecturally important: bootstrap.

A normal interval has a simple lifecycle:



Autoscroll, however, sometimes needs a temporary first interval and then a return to the canonical interval.

For example:



That is exactly what the bootstrap contract provides.

86. How Timer.bootstrap Actually Works

The Timer stores:

```
_delay  
_bootstrap  
_onTick
```

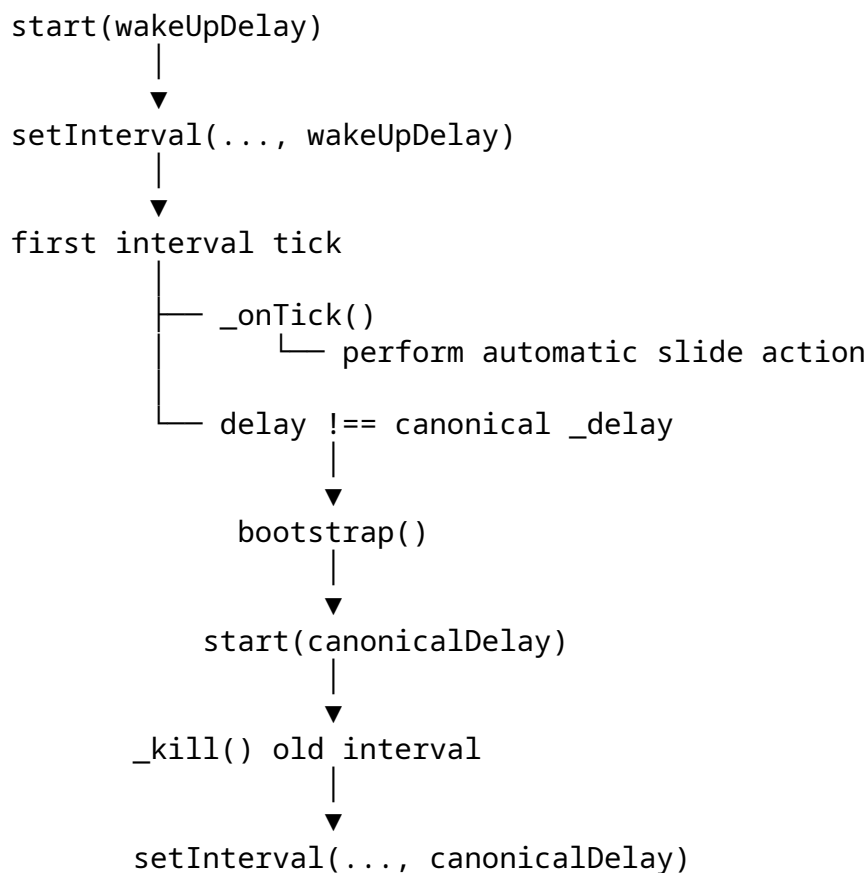
and `start(delay)` always creates the interval through:

```
_tick(delay)
```

The important part is:

```
_tick(delay) {  
    this._onTick();  
  
    if (this._bootstrap && delay !== this._delay) {  
        this._bootstrap(this._delay);  
    }  
}
```

The sequence is therefore:



So `bootstrap` is not a separate timer.

It is a **one-time transition from a temporary scheduling mode back to the normal scheduling mode**.

87. Why bootstrap Belongs to Timer, Not AutoscrollerSlider

A less reusable implementation could put this directly inside the slider:

```
AutoscrollerSlider
├─ start temporary timeout
├─ perform action
└─ restart normal interval
```

The current architecture instead makes Timer understand the generic mechanism:

```
Timer
├─ if first interval uses a temporary delay,
   bootstrap back to the canonical delay
```

while AutoscrollerSlider supplies the concrete bootstrap method.

Conceptually:

```
AutoscrollerSlider
├─ gives Timer:
   bootstrapMethod = start
   canonicalDelay
└─ ▼
   Timer
   └─ owns scheduling mechanics
```

This is a small example of dependency inversion at the mechanism level:

Timer does not know why the interval changes.
AutoscrollerSlider does not implement interval bookkeeping.

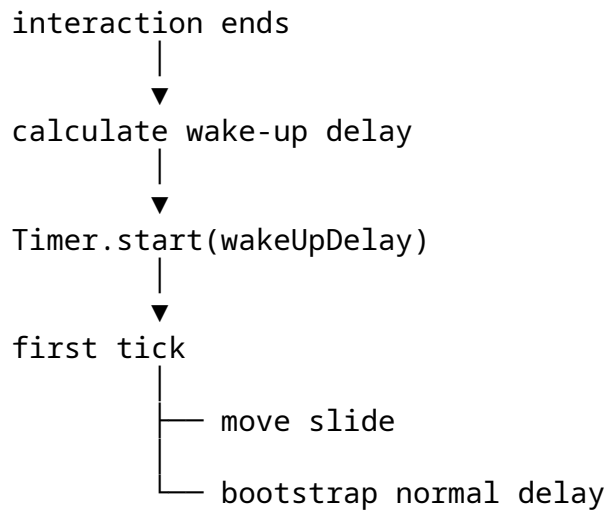
88. bootstrap and Adaptive Autoscroller

This becomes especially meaningful together with:

```
AUTOSCROLL_DELAY
AUTOSCROLL_WAKE_UP_DELAY
_getAdaptiveWakeUpDelay()
```

The flow is:

```
slide has been visible
├─ ▼
user interaction pauses autoscroller
├─ ▼
```



So the Timer is responsible for the scheduling transition, while the slider is responsible for deciding the appropriate temporary delay.

That is a particularly good example of infrastructure/domain separation.

89. Album End -> Slider -> slidechange -> AudioPlayer

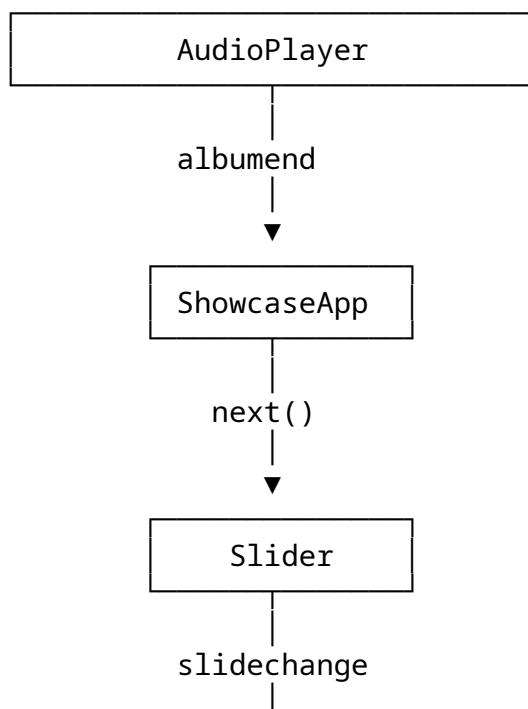
One of the most interesting architectural flows is the apparent loop between AudioPlayer and Slider.

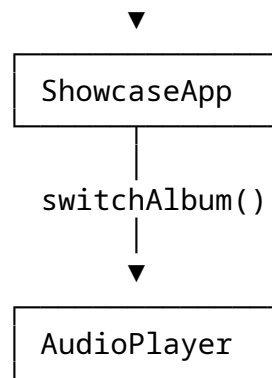
It is tempting to describe it as:

AudioPlayer → Slider → AudioPlayer

but that hides the important fact: there is no direct object-to-object call cycle.

The actual cycle is mediated by events and the application coordinator:





This is not recursive direct coupling.
It is an **event-mediated feedback loop**.

90. Why the Feedback Loop Is Not a Design Failure

The loop represents a real application rule:

```
album ends
  ↓
move to next visual item
  ↓
visual item identifies next album
  ↓
audio context follows the visual item
```

The important point is that each component only knows its own domain.

AudioPlayer:
"My album ended."

ShowcaseApp:
"When an album ends, advance the showcase."

Slider:
"My active slide changed."

ShowcaseApp:
"When the slide changes, synchronize the album."

The components never need to contain statements such as:

```
this._slider.next();
```

inside AudioPlayer or:

```
this._audioPlayer.switchAlbum(...);
```

inside Slider.

The application coordinator breaks direct coupling while still allowing the system to form a meaningful feedback cycle.

91. Album-End Event Is Cancelable

albumend is not just an informational event.

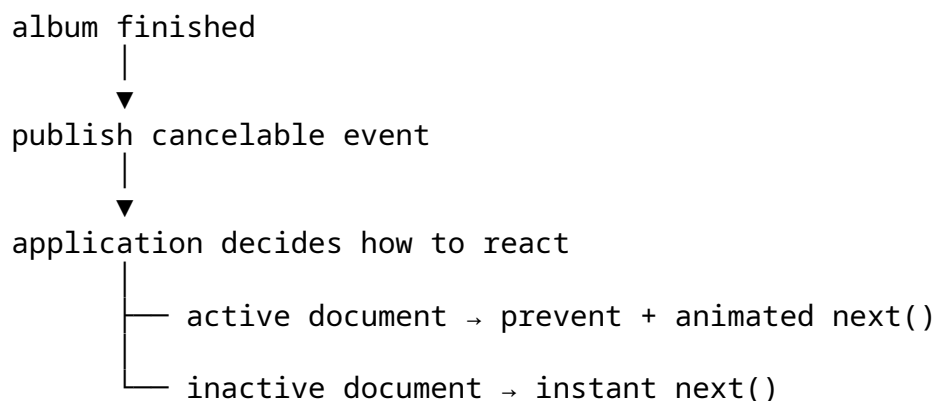
The AudioPlayer creates it as:

```
new CustomEvent("albumend", {
    detail: { index: ... },
    bubbles: true,
    cancelable: true,
});
```

and then checks:

```
return e.defaultPrevented;
```

This gives the event an additional control dimension:



So the event is simultaneously:

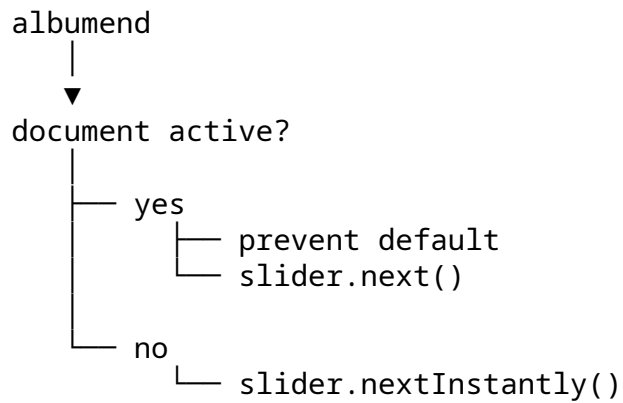
```
notification
+
coordination point
+
optional control signal
```

That is a subtle but important part of the architecture.

92. Active Document vs Background Document

The application distinguishes whether the tab/document is active.

That affects album-end navigation:



The reason is architectural as much as UX-related.

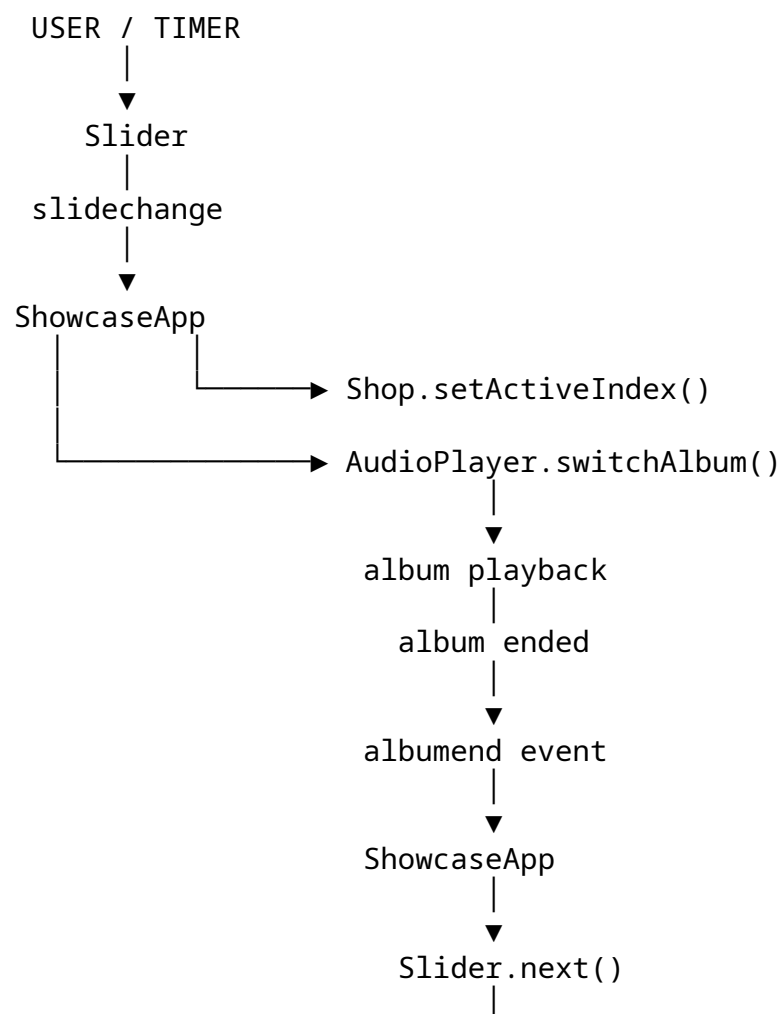
The visual transition is meaningful when the user is watching the page.

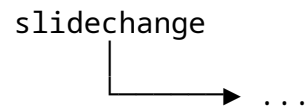
When the document is inactive, there is no reason to force an animated transition simply to preserve a visual sequence the user is not currently observing.

This is a good example of application policy staying in `ShowcaseApp`.

93. A More Complete Synchronization Loop

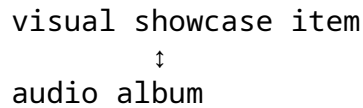
Combining the previous sections produces the full album-browsing loop:





The loop is intentional.

It creates synchronization between two independent representations of the same conceptual item:

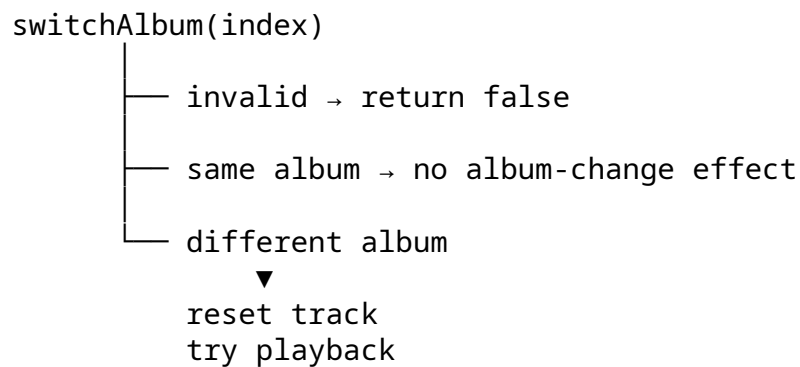


94. Guard Conditions Prevent Unwanted Cycling

The architecture does not create an uncontrolled infinite event cascade.

Several conditions act as guards.

For example:



Likewise:



So the feedback loop is event-driven but not self-amplifying.

This is one reason the earlier distinction between `slidemove` and `slidechange` is structurally important.

95. Album-End and the Separation of Intent

The `AudioPlayer` does not say:

"move the slider to album X"

It publishes:

"the album ended"

The application decides what that means.

Likewise, Slider publishes:

"the visual index changed"

and AudioPlayer decides how to interpret the corresponding album index.

This is a powerful form of semantic decoupling:

```
component event
  ↓
meaningful fact
  ↓
application policy
  ↓
semantic command on another component
```

96. Event Pipelines and Event Publication Are Different Mechanisms

Two mechanisms coexist:

PIPELINE

used mainly for incoming shared input

```
Event
  ↓
component A
  ↓
component B
  ↓
component C
```

and:

CUSTOM EVENT

used mainly for publishing a meaningful state change

```
component
  ↓
event
  ↓
ShowcaseApp
  ↓
application consequence
```

The distinction is useful:

pipe = "who gets a chance to handle this input?"

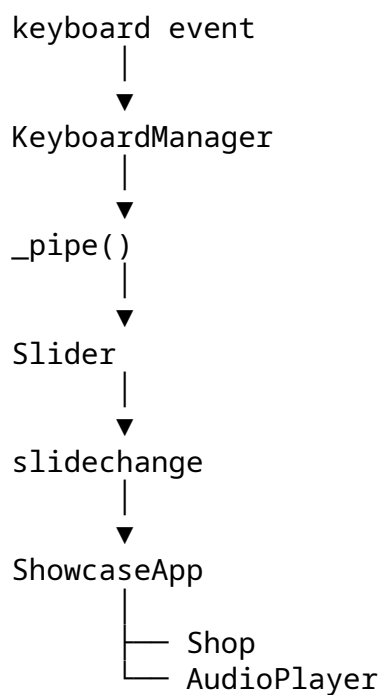
custom event = "something meaningful happened; who reacts?"

This keeps the architecture from treating every interaction as the same kind of communication.

97. One Interaction Can Cross Both Models

A user interaction can involve both mechanisms.

For example:



Thus:

incoming command path

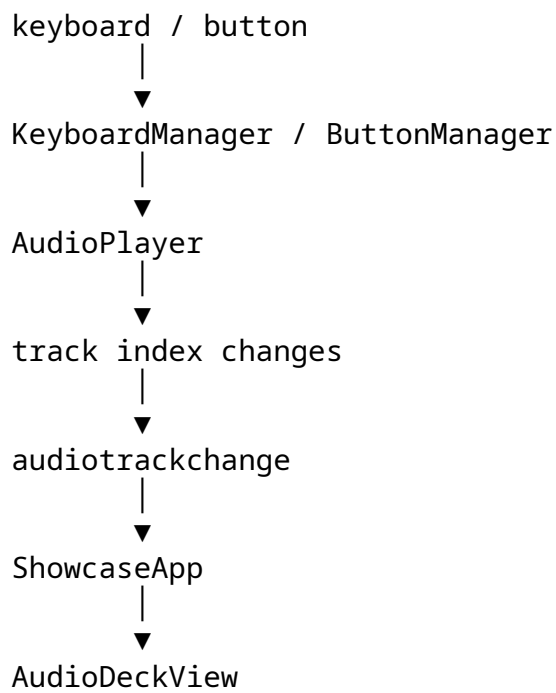
becomes:

outgoing domain-event path

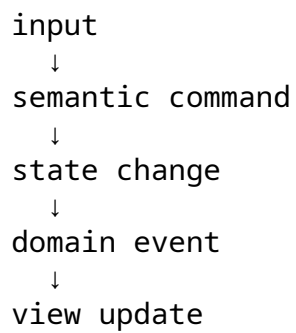
The two models are complementary.

98. Audio Track Change as a Similar Round Trip

The audio subsystem has its own version:



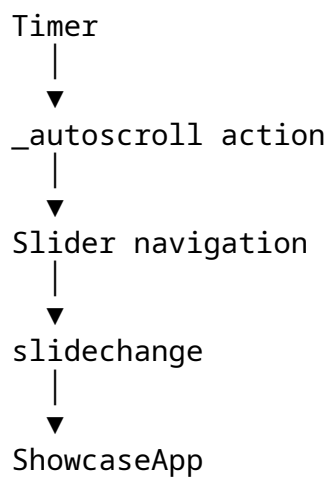
Again:



This is the recurring architectural shape of the whole system.

99. Timer and Event Architecture Meet at Autoscroll

Autoscroll is a particularly clear example of several subsystems cooperating.



└─ Shop
└─ AudioPlayer

Meanwhile the Timer itself knows nothing about:

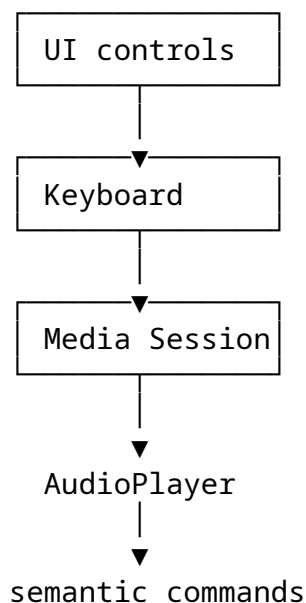
slides
albums
shop links
FSMs

It only schedules the callback.

This demonstrates why the infrastructure layer is useful even when a particular service is tiny.

100. Media Session as Another Input Boundary

The AudioPlayer accepts input from more than DOM keyboard/button handlers.



That reinforces the same principle:

External mechanisms should be translated into the owning component's semantic API as early as practical.

101. Why Preventing Default Browser Behavior Is Not the Whole Input Architecture

Some browser media shortcuts can come through mechanisms different from ordinary keyboard bubbling.

The application therefore treats:

keyboard event handling

and:

media-session handling

as related but distinct input channels.

The result is still unified at the `AudioPlayer` command boundary.

This makes the component more robust without requiring `ShowcaseApp` to know browser-media implementation details.

102. State Ownership Across the Feedback Loop

The album/slide loop contains three different state owners:

`Slider`

└─ active visual index

`AudioPlayer`

└─ current album / track / playback mode

`ShowcaseApp`

└─ coordination state such as moving/input routing

There is deliberately no:

global "current item" object

that every component mutates.

Instead:

local truth

+

events

+

coordination

keep the system synchronized.

103. The Architecture Is Event-Driven Without Being Globally Event-Based

Not every method publishes an event.

Not every operation goes through an `EventManager`.

Instead:

local method call

→ when the caller already owns the relationship

custom event

→ when a state/semantic fact should cross a boundary

pipeline

→ when shared input needs ordered handling

Choosing among these is part of the architecture.

That is more precise than calling the entire project simply “event-driven”.

104. Lifecycle Ownership

Several lifecycles are deliberately owned by different abstractions.

DOM subscription lifecycle

→ EventManager

keyboard action lifecycle

→ KeyboardManager

button action lifecycle

→ ButtonManager / Button

timer lifecycle

→ Timer

drag lifecycle

→ DraggableSlider

autoscroll lifecycle

→ AutoscrollSlider

audio lifecycle

→ AudioPlayer

cross-component lifecycle

→ ShowcaseApp

This table is useful because it explains why the system contains several small objects instead of one giant controller.

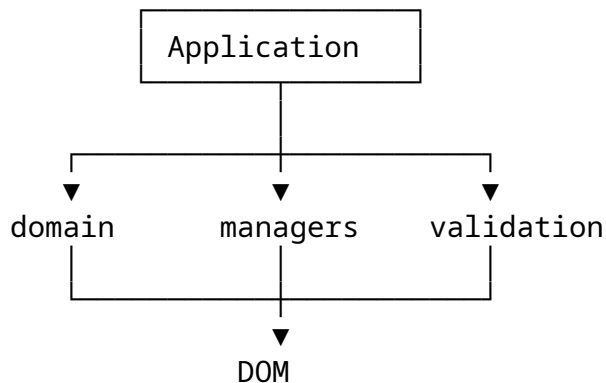
105. Why These Small Objects Form a Coherent Architecture

Individually, some abstractions look tiny.

For example:

Timer
Button
Shop
DOMValidator

The architectural value appears when their boundaries compose:



Each object removes one category of responsibility from somewhere else.
That is the criterion by which these abstractions should be judged.

106. A Note on the Project's Educational Evolution

The architecture was not produced as a purely theoretical design exercise.

The project evolved through implementation, testing and debugging.

That matters because some abstractions exist precisely because a concrete interaction exposed a real problem.

For example:

track movement
≠
committed slide change

became important when a small pointer movement could generate the wrong downstream behavior.

Likewise:

automatic navigation
≠
manual navigation

became important for autoscroll restart logic.

The architecture therefore reflects discovered behavior, not only predefined pattern diagrams.

107. Prototype/Class Parity as an Architectural Experiment

The second implementation is not merely a syntax rewrite.

It makes the same architecture an opportunity to examine what belongs to:

prototype inheritance

versus:

ES6 class syntax

The important parity target is:

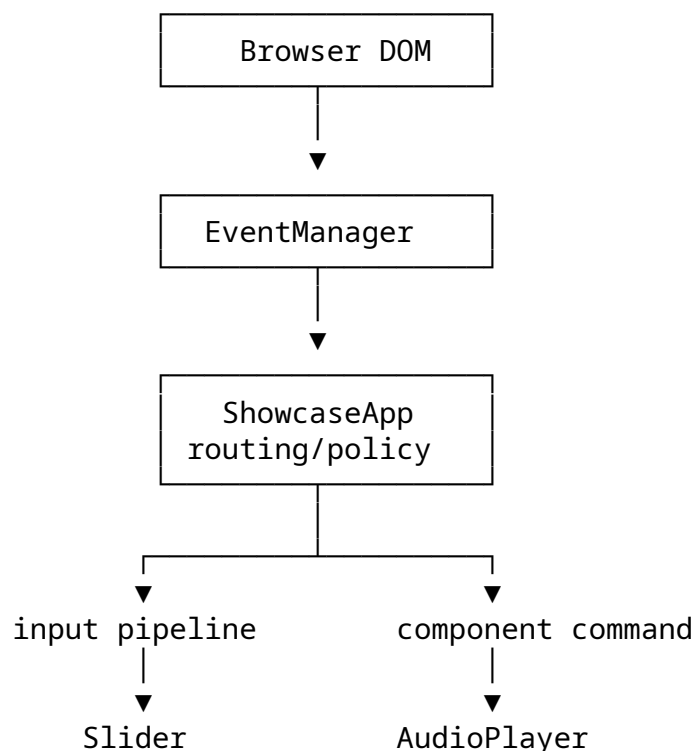
same responsibilities
same contracts
same FSM model
same event flow
same manager responsibilities
same component boundaries

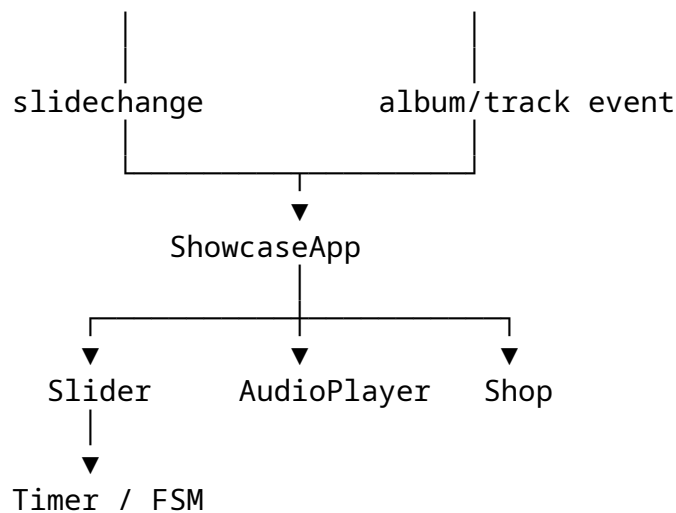
while the syntax differs.

This is why the documentation focuses on architecture rather than treating either syntax as the architecture itself.

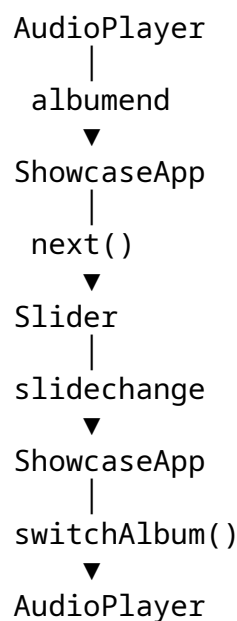
108. Final End-to-End Picture

The entire system can now be viewed as one large but understandable loop:





And the distinctive feedback loop is:



This is not accidental circular coupling.

It is a coordinated feedback loop in which:

events carry facts
 ShowcaseApp carries policy
 components carry domain state
 commands carry intent

That separation is one of the most important architectural achievements of the project.

109. Deep-Dive Closing Perspective

The most useful way to judge this architecture is not by counting classes or named patterns. Instead, ask whether every major responsibility has a clear owner and whether the system can explain a complete user interaction without hidden magic.

Here, the answer is largely expressed by the architecture itself:

input
↓
routing
↓
pipeline
↓
semantic command
↓
component algorithm
↓
local state
↓
domain event
↓
application coordination
↓
another semantic command

The project therefore demonstrates a broader lesson:

Architecture is not the collection of abstractions. Architecture is the set of boundaries that determines who owns a decision, who performs a mechanism, who announces a fact, and who is responsible for coordinating the consequences.

That is the deeper idea behind the implementation.